

LLM-HGAN: LLM-Enhanced Heterogeneous Graph Attention Networks for Advanced Persistent Threat Detection

Kun Lan¹, Gaolei Li², *Member, IEEE*, Wenkai Huang³, Jianhua Li⁴, *Senior Member, IEEE*, and Yantao Yu⁵, *Member, IEEE*

Abstract—Advanced Persistent Threats (APTs) have evolved into highly stealthy and organized cyber attacks, posing one of the most severe challenges to global cyber security. However, existing detection methods often suffer from high false positive rates and explosion of provenance graphs, making it difficult to counter sophisticated evasion techniques, diverse attack stages, and long latency periods. To address these challenges, this article proposes LLM-HGAN, a twostage APT detection framework that combines heterogeneous graph attention networks (HGAN) with large language models (LLMs). Specifically, suspicious processes are first identified by learning similarity on heterogeneous provenance graphs using LLM-HGAN, enabling the detection of subtle behavioral deviations across complex attack surfaces. Then, the detected anomalies are correlated with candidate attack paths and verified against a curated TTP (tactics, techniques, and procedures) rule base. Finally, LLMs operate in parallel to generate, normalize, and continuously update TTP rules from threat intelligence and system telemetry, ensuring that the detection process remains adaptive to evolving attacker behaviors. Extensive experiments on a hybrid dataset that combines curated APT samples from five public repositories with a highfidelity emulated APT scenario corpus show that LLM-HGAN achieves 95% detection accuracy, surpasses stateoftheart baselines by at least 14%, and reaches 98% accuracy on realworld APT variants, demonstrating strong robustness and generalization to unknownvulnerability attacks.

Index Terms—Heterogeneous graph attention network, large language models, provenance graph, TTP.

I. INTRODUCTION

ADVANCED Persistent Threats (APTs) epitomize an emergent cyber attack paradigm marked by institutionalized

organization, tactical stealth, persistent engagement, proactive adversarial maneuvers, and high-impact disruption, posing fundamental threats to global cyber infrastructure [1], [2], [3], [4], [5], [6]. Beyond large-scale service disruptions, APT campaigns also endanger data confidentiality and user privacy by enabling covert exfiltration of sensitive information from government agencies, enterprises, and critical infrastructure networks [7], [8]. Escalating geopolitical rivalries further drive state-sponsored APT actors to intensify cyber operations amid intensifying international competition. [9] shows the cyber kill chain model of APT attacks, as shown in Fig. 1.

In this model, the life cycle of an APT attack consists of seven key stages, including target reconnaissance, weapon construction, payload delivery, vulnerability exploitation, implant installation, command and control (C&C), and task execution. In December 2024, Russia-backed APT29 (Earth Koshchei) employed “malicious RDP” technology, utilizing forged remote desktop protocol configuration files to target global high-value entities. The attacks notably disrupted operations across Ukrainian government, military, and research institutions. Rapid and accurate APT attack detection in complex environments is a key research focus in global cybersecurity [10], [11]. This study analyzes recent APT incidents and technical reports, identifying emerging trends and key challenges in the current threat landscape:

- *Detection evasion techniques are becoming increasingly sophisticated:* APT groups are shifting toward fileless attacks to evade samplebased detection, employing attributionobfuscation techniques such as falseflag implantation via targeted IPs/services, C&C infrastructure concealment in webhosted environments, etc.
- *Attacks demonstrate progressively heightened complexity and diversification:* Modern APT groups increasingly adopt LivingOffTheLand tactics, leveraging trusted system tools. The expansion of opensource software has broadened supplychain attack surfaces, while the exploitation of zero-day vulnerabilities enables attacks beyond traditional APT frameworks.
- *The long-term and persistent nature of the attack process is more obvious:* Global APT groups distribute tailored Trojans via botnets. Postbreach, they conduct staged preparations—scoping environments, identifying targets,

Received 8 January 2026; revised 3 April 2026; accepted 20 April 2026. Date of publication 24 April 2026; date of current version 8 May 2026. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB3107702, in part by the National Natural Science Foundation of China under Grant 62572314 and Grant 62471301, and in part by Sichuan Science and Technology Program under Grant 2026YFHZ0223. Recommended for acceptance by Dr. Lan Zhang. (*Corresponding authors: Gaolei Li; Jianhua Li; Yantao Yu.*)

Kun Lan is with the China Electronics Technology Cyber Security Company, Ltd., Chengdu 610041, China, and also with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: 2331lankun@sjtu.edu.cn).

Gaolei Li, Wenkai Huang, Jianhua Li, and Yantao Yu are with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: gaolei_li@sjtu.edu.cn; sjtuhwk@sjtu.edu.cn; lijh888@sjtu.edu.cn; yantaoyu@sjtu.edu.cn).

Digital Object Identifier 10.1109/TNSE.2026.3687554

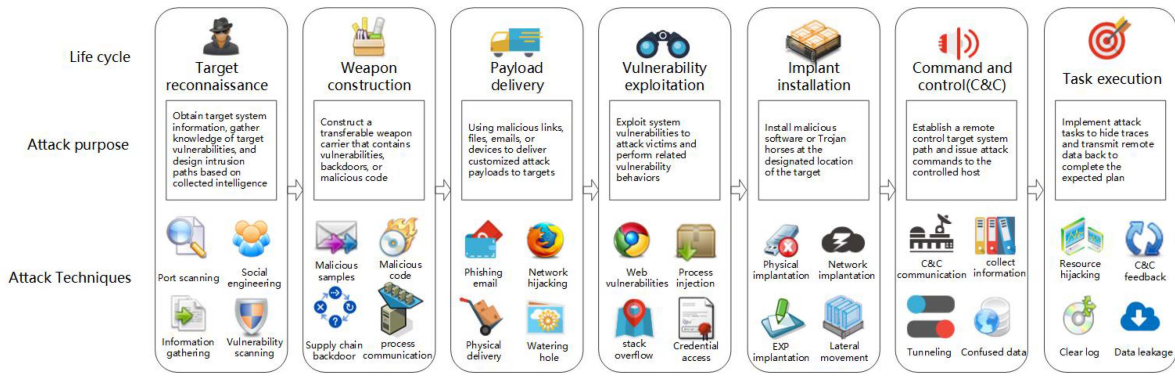


Fig. 1. Illustration of the APT attack lifecycle, depicting major attack stages, employed techniques, and ultimate objectives.

and disabling security controls—to ensure reliability, while evading sandboxes and manipulating vectors to prolong operations.

In recent years, APT detection research has remained active [12], [13], [14], [15], [16], [17], [18], [19], [20]. Chen et al. (2025) introduced a multi-agent deep reinforcement learning (MARL) framework to address the inherent uncertainty and dynamic adaptability of adversarial behaviors [17]. Xiao et al. (2025) proposed an RL-based APT defense scheme to optimize both the continuous scan interval of metering data and repair rate to mitigate the potential loss for meter data management systems (MDMSs) in smart grids with large-scale meters [18]. From the perspective of attack sample generation, Cheng et al. (2025) proposed a system called TAGAPT, which can automatically generate a large number of APT samples with source-level granularity [19]. Moothedath et al. (2024) proposed an information flow tracking game-theoretic framework for resource-optimized APT detection [20]. A prominent research focus involves transforming system-generated logs and alerts into provenance graphs to facilitate graph-theoretic APT attack detection [21], [22], [23]. Contemporary approaches face two fundamental limitations: detecting multi-year incubating APTs necessitates excessive provenance graph storage and manual rule updates, severely compromising operational feasibility; sophisticated evasion tactics markedly degrade attack visibility during lateral movement and data exfiltration by minimizing exposure durations and obfuscating behavioral signatures. Rule-based packet inspection exhibits high false positives in high-throughput networks, as benign traffic frequently adheres to legitimate communication patterns. Real-time detection mandates immediate alert generation during active communication sessions. Existing technical approaches cannot perform complex, computationally intensive analyses within tight time constraints, severely compromising the efficacy of packet-based detection methodologies.

To address these challenges, this study presents LLM-HGAN, a novel detection framework that integrates HGAN and LLM to enhance the effectiveness of APT detection. By leveraging twostage decision theory from systems science, LLM-HGAN decomposes complex APT detection into sequential phases, representing a clear methodological departure from conventional approaches. The first stage detects anomalous behaviour, aiming to initially exclude non-complex attack anomalies to narrow the attack detection scope. The technical approach combines provenance graph analysis, similarity learning, and anomaly

detection technology based on HGAN. The second stage detects attack activities. The technical approach involves clustering anomalous processes to reconstruct attack phases, employing an enhanced forward traversal algorithm to extract complete attack paths from the provenance graph. These paths are then matched against meticulously maintained TTP rules to achieve precise APT attack identification. Simultaneously, the LLM module continuously generates and updates the TTP knowledge base, ensuring the detection framework adapts to evolving attacker behavior. Crucially, second-stage TTP generation need not be concurrent with first-stage anomalous behavior processing, greatly enhancing efficiency and feasibility. HGAN outperforms traditional graph neural networks like Recurrent Graph Neural Networks, Convolutional Graph Neural Networks, and Graph Autoencoders in detecting APTs. Its advantages stem from heterogeneous modeling, meta-paths, multi-level attention, semantic comprehension, and relational reasoning, enabling it to uncover hidden correlations and patterns in extensive datasets. Traditional graph convolutional network (GCN) are inherently limited to graphs with uniform node and edge types. In contrast, provenance graphs derived from log data are inherently heterogeneous, comprising diverse system entities and interaction relations. HGAN leverages this heterogeneity to selectively emphasize critical information, suppress irrelevant interference, and refine the graph structure, thereby achieving superior performance in detecting complex, stealthy, and multidimensional attack behaviors.

Previous research such as [42] and [43] employed HGANs to detect APT attacks but did not involve LLMs. Meanwhile, studies [47] and [48] mentioned using LLMs for APT attack detection but did not incorporate graph attention networks. To the best of our knowledge, the uniqueness of LLM-HGAN lies in its pioneering integration of LLMs with HGANs. On one hand, LLMs enable automated updates to TTPs, thereby achieving dynamic enhancement of LLM-HGAN itself. On the other hand, HGANs help control the scale of the provenance graph. Consequently, this approach significantly enhances detection capabilities against APTs from two distinct perspectives. Given that LLMs are primarily architected for natural language processing, their unmediated application to attack threat detection presents significant limitations. Although HGAN demonstrate unique merits in regulating the scale of attribution graphs, traditional methodologies are often burdened by a reliance on hand-crafted TTPs. This integration, therefore, represents more than an

engineering task; it is a principled approach to amalgamating the respective capabilities of each model, aligning them with the distinct phases of APT attack detection to achieve a synergistic effect. The study identifies a research gap in understanding the familial characteristics of APT attacks, underscores the need for long-term dynamic adjustment mechanisms in future detection methods, and introduces three key innovations:

- To address the growing technical challenges posed by increasingly evasive APT detection techniques, the LLM-HGAN model employs a LLM to integrate advanced TTP rules from multiple global sources. These include updates from the MITRE ATT&CK framework, threat intelligence platforms, open-source intelligence communities, and internal enterprise attack data. This integration enables in-depth characterization of adversarial strategies such as file-less operations, false-flag spoofing, and C&C obfuscation.
- Aiming at the problem of APTs are becoming more complex and diverse. The LLM-HGAN framework initially constructs an anomalous process detection model by integrating provenance graphs, HGAN, and similarity learning, followed by TTP-based behavioral matching to identify APT activities. This approach enables dynamic discovery of novel attack entities, behavioral patterns, and execution trajectories.
- To address APT attacks' prolonged duration, LLMHGAN integrates large language models' continuous learning of the latest attack features with the HGAN mechanism. This combination effectively manages APT provenance graph scale, enabling dynamic and longterm detection efficacy.

The rest of this paper is organized as follows. Section II presents related work, Section III introduces TTP and provenance graph, Section IV describes the LLM-HGAN model, Section V describes the experiments conducted on the LLM-HGAN model. Section VI summarizes the study and proposes the future research plan.

II. RELATED WORKS

The literature relevant to LLM-HGAN primarily falls into three domains: provenancebased analysis, HGAN and LLM for attack detection.

A. APT Attack Detection Based on Provenance Graphs

Akbar et al. (2023) used a provenance graph to detect anomalous host node behavior [24], but this method was weak in detecting zero-day APT attacks, while the LLM-HGAN method demonstrated strong capabilities. Aly et al. (2024) proposed MEGRAPT, a provenancegraphbased APT detection framework. Its batch kernel log processing reduces graph diversity and detection efficacy [25]. In contrast, LLMHGAN constructs provenance graphs directly from audit log sequences, offering greater adaptability. Chen et al. (2022) developed APT-KGL, a provenance and GNNbased detector targeting singlepoint attacks [26]. LLMHGAN, in contrast, is designed for complex, multistage network attacks. Wang et al. (2022) proposed THREATTRACE [27], a GraphSAGE-based anomaly detection framework, but it ignores practical storage scalability, while LLM-HGAN focuses on dynamic provenance graph storage.

Irshad et al. (2021) developed TRACE, a static analysis tool that limits provenance tracking to internal program-unit dependencies [28]. Its reliance on outdated data hinders practical use, whereas LLM-HGAN leverages large-scale models for intelligent analysis with enhanced scalability. Li et al. (2024) proposed T-trace, building event provenance graphs via log correlation/tensor decomposition [29] to target active APT attacks (visible impact); LLM-HGAN detects latent APT behaviors (no significant damage). Wu et al. (2023) proposed Paradise, a real-time distributed intrusion detector using provenance dependencies for pruning/extracting process feature vectors [30]. LLM-HGAN skips pruning the attribution graph to retain hidden APT attack clues. Kurniawan et al. (2022) introduced KRYSTAL, which integrates threat detection, attack graphing, scenario reconstruction, and RDF analysis [31]. Its effectiveness diminishes against APT attacks that evade known rules, whereas LLMHGAN targets unknown attack detection. [32][33] expanded tag semantics with APT behavior-state semantics, refined rules, and incorporated complex semantics/external knowledge. Though enhancing expressiveness, this complexity complicates implementation. LLM-HGAN integrates provenance graphs and TTP rules algorithmically for improved usability. Alsaheel et al. (2021) proposed ATLAS, extracting attack/non-attack sequences from attribution graph entities for learning [34], but LLM-HGAN avoids ATLAS's overfitting/underfitting issues. Yang et al. (2023) developed PROGRAPHER for anomaly detection; however, its dependence on a manually predefined node count limits practicality [35], LLMHGAN overcomes this constraint. Xu et al. (2022) introduced DEPCOMM, a graph summarizer that compresses dependency graphs through processcentric community partitioning and structural summarization [36]. This approach, however, involves complex multidimensional balancing in path scoring, whereas LLMHGAN reduces such module dependencies. Akbar et al. (2023) proposed APT detection using data provenance and metric learning [37], but the reliance on manual feature extraction for online adaptive training limits its practicality. In contrast, LLM-HGAN leverages LLM to automatically generate TTPs, offering flexibility and high efficiency.

Provenance-graph-based APT detection, while critical, faces challenges including interference from irrelevant graph elements and the computational burden of large-scale features in long-term attack scenarios.

B. APT Attack Detection Based on HGAN

Zhou et al. (2024) proposed PANNER, a POS-aware nested named entity recognition model using dilated random walks on grammatical paths to build heterogeneous graphs [38]. However, PANNER has low accuracy, while LLM-HGAN outperformed others in experiments. Li et al. (2021) introduced dual GNNs, using metapath aggregation for provenance graphs and edge enhancement for host graphs to model APT behaviors across network and system levels [39]. While reliant on combined environmental data, this differs from LLM-HGAN, which detects unknown attacks without such constraints. Duan et al. (2023) proposed a semi-supervised intrusion detection method using dynamic graph neural networks (DLGNN) to convert

network traffic into spatiotemporal graphs [40]. Liu et al. (2022) introduced FewM-HGCL, a self-supervised framework for detecting malware variants using heterogeneous graph contrastive learning, enabling graph-instance-based discrimination [41]. Zhang et al. (2025) proposed a preprocessing method for network logs to establish an APT detection model based on an improved graph convolutional network [42]. Research on HGAN for detecting APT attacks is in its early stages, with reliance on graph attention networks without TTP knowledge posing challenges. Wu et al. (2025) proposed an efficient intrusion detection via HGAN and parallel provenance analysis [43]. However, unlike this approach, LLM-HGAN does not utilize parallel provenance analysis to detect intrusions. [44], [45], [46] mentioned methods using graph attention networks to detect network anomalies, but these approaches differ significantly from those used for APT attack detection and LLM-HGAN. Ghafir et al. (2018) introduced a supervised machine learning-based APT detection system (MLAPT), which demonstrates limited adaptability to anti-analysis techniques in modern APT attacks [89]. In contrast, the LLM-HGAN approach proposed herein integrates graph attention networks with large language models (LLMs), facilitating efficient and flexible adaptation to the evolving landscape of APT attacks. Presekal et al. (2024) developed an EGC-LSTM-based approach for APT detection in cyber-physical power systems, which depends extensively on domain-specific network features. In contrast, the proposed LLM-HGAN offers greater generalizability by eliminating the requirement for such specialized scenario-dependent characteristics [90].

C. Leveraging LLM for APT Detection in Complex Networks

Benabderahmane et al. (2025) proposed APT-LLM, an embedding-based anomaly detection framework combining LLMs (BERT, ALBERT, DistilBERT, RoBERTa) with autoencoder architectures for APT detection [47]. Xu et al. [48] introduced APTSniffer, an encrypted APT traffic detection model that leverages fewshot inference and generalization capabilities of LLMs by converting raw traffic data into natural language inference instances. Selim et al. [49] presented a proof-of-concept validation of LLM for cyber attack detection in smart inverters via textual control commands. He et al. [50] proposed a collaborative vehicular threat-sharing framework leveraging vehicular honeypots to aggregate threat data for fine-tuning LLMs, thereby enhancing IoV security. LLM-based APT attack detection remains in its conceptual infancy or is confined to specific scenarios. Yu et al. [51] proposed a Label-free Detection against APTs in Edge Networks via LLM and GCN. GOLEC et al. [52] presented the first systematic review and taxonomy for LLM-assisted APT detection in 6G networks, this method is primarily designed for 6G wireless networks. HMIMOU et al. [53] proposed a Multi-Agent System for Cyber security Threat Detection and Correlation Using LLM, this method requires multiple agents and is still in the model exploration phase, where its maturity is not yet high. Zuo et al. [54] presented a LLM-based semantic enhancement method for APT detection without involving specific detection techniques. Leopoldo et al. [55] proposed an

LLM-based cyberattack detection approach utilizing network flow statistics, though its detection accuracy warrants further improvement. Directly converting large-scale network traffic into natural language for LLM input presents practical feasibility challenges.

In summary, existing provenance-graph-based approaches are limited by irrelevant noise and scalability issues, while current HGAN-based methods primarily focus on structural modeling without integrating TTP semantics. Current LLM-based APT detection methods are often conceptual or domain-specific, facing scalability and heterogeneous data processing challenges. To overcome these limitations, this research proposed LLM-HGAN framework jointly leverages HGAN for fine-grained abnormal process detection and LLM for dynamic TTP rule generation, thereby achieving accurate, adaptive, and scalable APT detection across diverse attack scenarios. Notably, LLM-HGAN utilizes the continuous learning capability of LLM to effectively mitigate APT attacks exploiting zero-day vulnerabilities and family-based variants. Experiments based on real network traffic demonstrate that the proposed scheme outperforms existing methods. Yang et al. (2025) proposed an LLM-based APT detection framework (LLMAPTDS) but did not address key aspects such as attack attribution graph generation [91]. In contrast, LLMHGAN introduces a comprehensive twostage detection methodology.

III. PRELIMINARIES

A. Special Characteristics of APT Attack Provenance Graph

A provenance graph is a bipartite structure with system entities as nodes and system events as directed edges, representing temporal and typological relationships. Through the systematic analysis of over 300 documented APT attack cases, this study empirically demonstrates that data-centric operational patterns (e.g., anomalous file access activities) persistently manifest within system log traces across all attack phases, irrespective of the sophistication level of the intrusion. Quantitative analysis identifies substantial process similarity among APT family attacks, which can be formally modeled via set theory and process algebra (1).

$$A = \{S_1, S_2, \dots, S_n\} \cup \{P_1, P_2, \dots, P_n\},$$

- 1) Temporal relationship: $\forall S_i, \exists S_k \mid S_i \rightarrow S_k$,
- 2) Process mapping: $\exists f : S \rightarrow 2^\Pi$,
- 3) Completeness: $\bigcup f(S_i) = \{P_1, \dots, P_m\}$,
- 4) Attack complexity: $C = f(|S|, |P|)$, $f : S \rightarrow \mathbb{R}^+$,
- 5) Collaborative relationship: $\exists P_\alpha, P_\beta \in \Pi$, $P_\alpha \leftrightarrow P_\beta$,
- 6) Attack chain inevitability: $A \neq \emptyset \Rightarrow (\exists S) \wedge (\exists P)$.

(1)

In the above equation, A denotes an APT attack activity, where $S_i \in \Sigma$ represents an attack step (with Σ being the space of all possible attack steps) and $P_j \in \Pi$ denotes a process (with Π being the system's entire process space). Moreover, $n \geq 1$ indicates that at least one attack step must be included, and $m \geq 1$ ensures that at least one process is involved.

TABLE I
FUNCTIONAL MODULES OF LLM-HGAN MODEL.

Composition modules of the LLM-HGAN model	Implemented functions	Possible deployment locations
HGAN-based anomaly detection module	Detecting abnormal processes from Provenance graph using HGAN and similarity learning methods.	Terminal devices
TTP-based attack detection module	Utilising TTP to detect APT attack behaviour based on clustering of abnormal processes. Use LLM to dynamically update TTPs from multiple sources such as APT attack analysis reports, blog forums, academic papers, etc.	Network equipments or terminal devices
LLM-based TTP generation module		Cloud or server-side

B. Basic Characteristics of TTP

TTP characteristics encapsulate semantic descriptions of APT attackers' tactical intent, technical capabilities, and procedural execution, reflecting their strategic methodologies during operations. Operational constraints that slow tool and personnel updates make TTPs more stable than IOCs within specific threat groups. Consequently, TTP-based detection is more effective for identifying APT activities. Notably, according to [56], APT attackers employ diverse attack technique combinations to execute the same tactical objective, yet APT attack families generally utilize highly similar attack techniques and paths. While prior methodologies have incorporated cyber threat intelligence (CTI) and knowledge bases, the proposed LLM-HGAN framework leverages LLMs to enable continuous, automated learning of evolving threat intelligence. This approach significantly diminishes dependence on labor-intensive manual analysis inherent in conventional techniques. Furthermore, it introduces a novel ensemble learning framework to systematically mitigate accuracy-related challenges.

IV. MODELS AND METHODS

A. The Basic Principles of LLM-HGAN

LLM-HGAN integrates HGAN and LLM, which will enable more flexible and efficient capture of hidden correlations and patterns, thereby significantly improving the ability to discover complex APT attack chains. LLM-HGAN involves two detection stages: abnormal process detection (Stage I) and attack activity detection (Stage II). LLM-HGAN employing three modules: HGAN-based anomaly detection, TTP-based attack detection, and LLM-based TTP generation (Table I), with its workflow illustrated in Fig. 2.

APT attacks now feature more sophisticated attack/defense tactics, marked by longer latency and stronger stealth. The two-stage decision theory decomposes such processes into two sequential and logically interconnected stages to achieve analytical optimization: anomaly identification then malicious activity

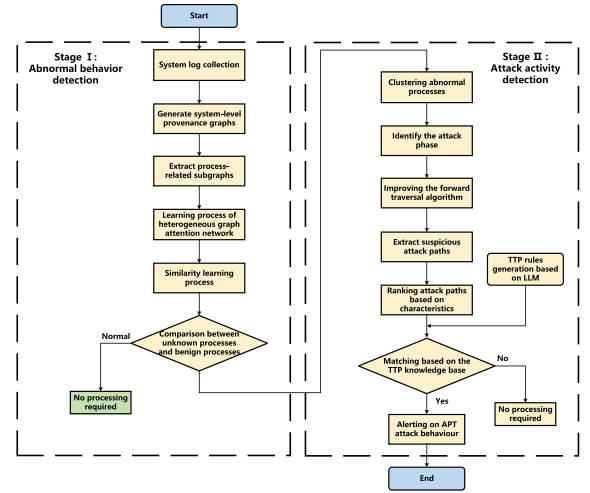


Fig. 2. Workflow between functional modules of the LLM-HGAN model.

recognition, aligning with (1)'s logic. It is effective, particularly for detecting family-based APT attacks:

Stage I: LLM-HGAN constructs provenance graphs from system logs, employs subgraph extraction with similarity learning and HGAN to identify anomalous processes through comparison with benign baselines.

Stage II: LLM-HGAN clusters anomalous processes from Stage I and matches them with TTP-based attack patterns generated by the LLM to identify attack stages. An improved forward traversal algorithm extracts the attack path, and feature-based ranking is applied to detect APT attack behavior.

From an algorithmic perspective, the LLM-HGAN model consists of three functional modules: an HGAN-based anomaly detection module, a TTP-based attack detection module, and an LLM-based TTP generation module, with their functions and deployment locations detailed in Table I.

LLM-HGAN optimizes resource allocation via modular decoupling of three core components: anomaly detection, attack detection, and LLM-driven TTP analysis. This separation yields two benefits: i) reduced computational overhead, and ii) improved dynamic adaptability of the LLM-based infrastructure.

The first stage completes fine-grained anomaly detection, while the second stage combines TTP rules matching to further detect multi-step APT attack behaviour. Introducing the mechanism of LLM constantly updating TTP in the two-phase detection mode will allow more attack clues to be detected, thus detecting early APT attack behaviors.

The long-term learning capability of LLM-HGAN refers to the capacity of LLMs to continually understand and process TTPs in the ATT&CK framework. This is achieved through their advanced language understanding, contextual reasoning, pattern generalization, and knowledge association capabilities. By leveraging extensive pre-trained security knowledge, semantically modeling natural language descriptions, and employing flexible in-context learning mechanisms, these models can effectively identify, generate, reason about, and simulate TTP-related behaviors.

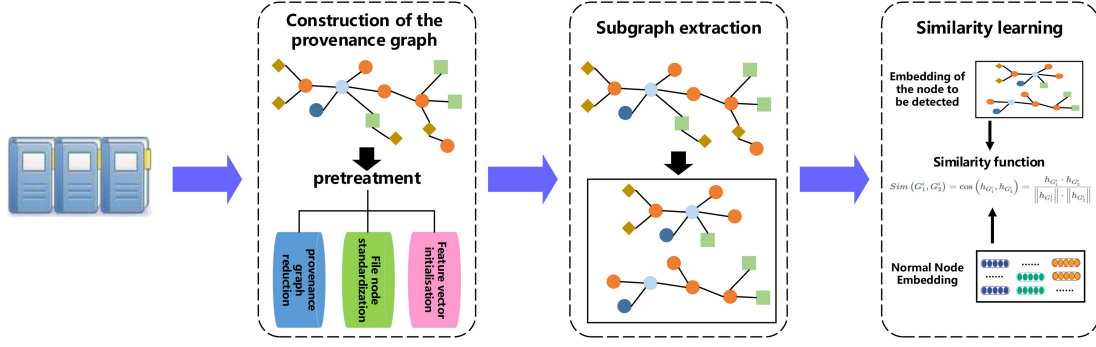


Fig. 3. LLM-HGAN driven anomaly detection framework.

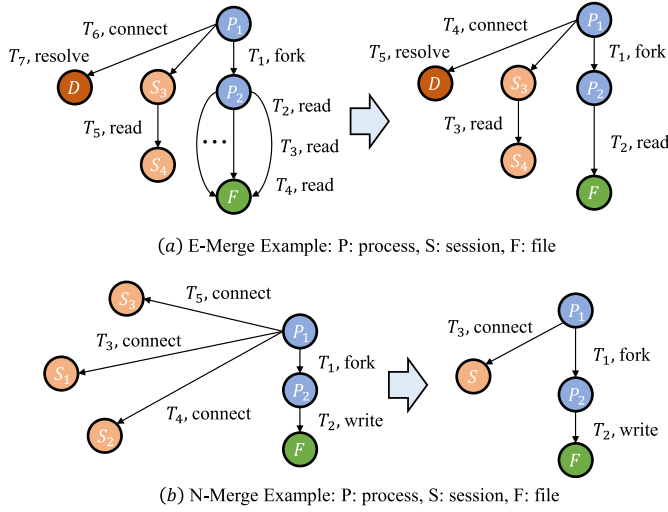


Fig. 4. E-Merge Example and N-Merge Example.

B. HGAN-Based Anomaly Detection Module (Stage I)

Provenance graphs contain heterogeneous entities (e.g., processes, files, network connections) with distinct roles in system activities and attacks. LLM-HGAN integrates HGAN to capture node-weighted graph features and extract contextual interaction info from process nodes, enabling precise malicious process identification and reducing APT detection false positives. Meanwhile, it detects malicious processes by analyzing behavioral similarities between unknown and known benign processes, without needing prior APT attack knowledge (Fig. 3):

The HGAN-based anomaly detection module of LLM-HGAN mainly consists of three parts: Construction of provenance graph, subgraph extraction, and similarity learning.

LLM-HGAN models a provenance graph as $G = (V, E)$, where V is the set of nodes and $E \subseteq V \times V$ is the set of directed edges. An event is represented as $e = (A_{src}, r, B_{sink})$, where $A_{src} \in V$ denotes the initiator (source node), r the event type, and $B_{sink} \in V$ the recipient (sink node). The provenance graph is constructed following Algorithm 1 and implemented using NetworkX.

System-log-derived provenance graphs effectively record host activities, but their large size leads to dependency explosion

Algorithm 1: Construction of provenance graph.

```

1: Input:  $D$  ▷ System audit log
2: Output:  $G = (V, E)$  ▷ Provenance graph
3: Init:  $V \leftarrow \emptyset, E \leftarrow \emptyset$ 
4: For each  $e \in D$  do
5:   if  $B_{sink} \notin V$  then
6:     create new node  $B'_{sink}$ 
7:      $V \leftarrow V \cup \{B'_{sink}\}$ 
8:   if  $A_{src} \notin V$  then
9:     create new node  $A'_{src}$ 
10:     $V \leftarrow V \cup \{A'_{src}\}$ 
11:   end if
12:    $E \leftarrow E \cup \{(A_{src} \rightarrow B_{sink})\}$ 
13: else
14:   if  $A_{src} \notin V$  then
15:     create new node  $A'_{src}$ 
16:      $V \leftarrow V \cup \{A'_{src}\}$ 
17:   end if
18:    $E \leftarrow E \cup \{(A_{src} \rightarrow B_{sink})\}$ 
19: end if
20: end for

```

in complex multi-phase APT attacks. This situation requires compression techniques to reduce computational demands while maintaining detection effectiveness. Additionally, noise from benign events complicates analysis. To tackle these issues, LLM-HGAN utilizes two compression strategies: E-Merge and N-Merge.

E-Merge: Multiple parallel edges between process and file/network nodes in provenance graphs indicate frequent short-interval read/write operations from repeated system calls under resource constraints. As [57] shows, such edges offer little additional value for APT detection, but merging them improves graph readability and analyzability. Merging requires: i) edge type homogeneity, ii) identical associated node types, and iii) temporal proximity below a predefined threshold α , as illustrated in Fig. 4(a).

N-Merge: Prior studies [58], [59] show that provenance graphs contain numerous read-only files, such as libraries, configuration files, and process initialization resources (e.g., `/lib64/libdl.so.2`). Merging nodes of the same type into a

TABLE II
FEATURE VECTOR OF NODES.

Name embedding	Type	Was-Generate-By	Used	Was-Triggered-By	Was-Derived-From
process	0	3	0	4	1
file	1	2	0	0	1

single node preserves the semantics of a process's core system activities, as illustrated in Fig. 4(b).

Diverse user types cause parameter naming confusion, yet file names and similar attributes lack practical significance for APT detection. LLM-HGAN standardizes file-type node names while preserving semantic information, grouping them into file/network categories. File class node names are uniformly represented as: system_file, lib_file, programs_file, user_file. Network class node names are uniformly represented as: ip_address, domain, url, connection, session.

For each file-type node in the provenance graph, the node name is normalized to its corresponding entry in the predefined vocabulary. For example, /user/secret.pdf is mapped to user_file. To address provenance graph heterogeneity, LLM-HGAN decomposes node initialization feature vectors into two components: identity descriptors and structural attributes. Since process names remain static during host operation whereas APT processes typically adopt random identifiers (e.g., Linux attackers generating malware with random strings in /tmp), the framework instead leverages execution paths as unique process identifiers.

System entities show stable distribution and regular neighbor-access patterns. Notably, provenance graphs from compromised systems deviate significantly from benign ones, especially in malicious processes' abnormal access frequency to adjacent nodes. Thus, during preprocessing, LLM-HGAN systematically calculates per-node edge-type frequencies and integrates these metrics with node metadata and type classifications to build comprehensive node feature vectors. Results are summarized in Table II:

Name embeddings encode node identifiers, type embeddings specify node categories, and relationships such as Was Generated By and Used quantify edge-type frequencies. Experiments show LLM-HGAN cuts APT success rates (using file deletion/concealment in benign files) by at least 50%, thanks to induced anomalies in file access/generation/deletion patterns detected by fine-grained execution-path tracing.

LLM-HGAN uses subgraphs to represent the activity information of APT attack processes. Given a process node v_p , the LLM-HGAN methodology employs (2) as the foundational computational framework for subgraph sampling, specifically designed to extract a k -order subgraph centered on a target process node v_p from the constructed provenance graph $G = (V, E)$.

$$SG^{(k)}(v_i, G) = \begin{cases} \{v_j | (v_i, v_j) \in E\}, & k = 1 \\ \{SG^1(v_z, G) | v_z \in SG^{(k-1)}(v_i, G)\}. & k > 1 \end{cases} \quad (2)$$

Algorithm 2: Generation of P-pair graph data.

```

1: Input:  $SG = \{SG_1, SG_2, \dots, SG_n\}$   $\triangleright$  Set of attack nodes  $A$ 
2: Output: P-pair of graphs  $T$ 
3: Normal subgraph set:  $S \leftarrow \emptyset$ , Malicious subgraph set:  $M \leftarrow \emptyset, T \leftarrow \emptyset$ ;
4: For each  $SG_i : SG$  do
5:   if  $SG_i \in A$  then  $\triangleright$  contains attack nodes
6:      $SG_i$  assigns label  $y_i = 1$ 
7:      $M = M \cup (SG_i, y_i)$ 
8:   else
9:      $SG_i$  assigns label  $y_i = 0$ 
10:     $S = S \cup (SG_i, y_i)$ 
11:   end if
12: end for
13: For each  $e_i : (SG_i, y_i) \in S$  do
14:   For each  $e_j : (SG_j, y_j) \in M$  do
15:     if  $SG_i$  and  $SG_j$  belong to the same class of processes using subgraphs then
16:       if  $y_i == y_j$  then
17:         The label  $y = 1$  for the subgraph pair  $SG_i$  and  $SG_j$ 
18:          $T \cup \{(SG_i, SG_j), y\}$ 
19:       else
20:         The label  $y = -1$  for the subgraph pair  $SG_i$  and  $SG_j$ 
21:          $T \cup \{(SG_i, SG_j), y\}$ 
22:       end if
23:     end if
24:   end for
25: end for

```

Research indicates that setting the k value for subgraph sampling in the provenance graph to 2 ensures system stability during APT attacks. Parameter values exceeding 3 may cause exponential inflation, resulting in weak correlations between numerous nodes and increased computational costs. After subgraph sampling, the set of subgraphs of all process nodes in the provenance graph was obtained $SG = \{SG_1, SG_2, \dots, SG_n\}$. The training set data must be labeled for each subgraph for model training. The training set data will contain the corresponding attack entities {B,C}. If the sampled subgraph contains attack entities, the label will be 1; otherwise, it will be 0.

To address imbalanced attack/benign subgraph distributions, LLM-HGAN uses similarity learning to integrate dual subgraph features, boosting training efficiency by prioritizing structural over process-level attributes. This method requires paired subgraph inputs: Algorithm 2 generates labeled subgraphs (1=attack-containing, 0=benign), then pairs them as 1 (same process class) or -1 (cross-class) by origin.

The similarity learning method of LLM-HGAN converts paired subgraphs (G_1^1, G_2^1) and (G_1^2, G_2^2) into graph sequences $(G_1^1, G_2^1, G_1^2, G_2^2)$, which are used as inputs for the same HGAN, outputs the corresponding feature vector h_v . Then, in HGAN, the feature vectors corresponding to the subgraphs are fused

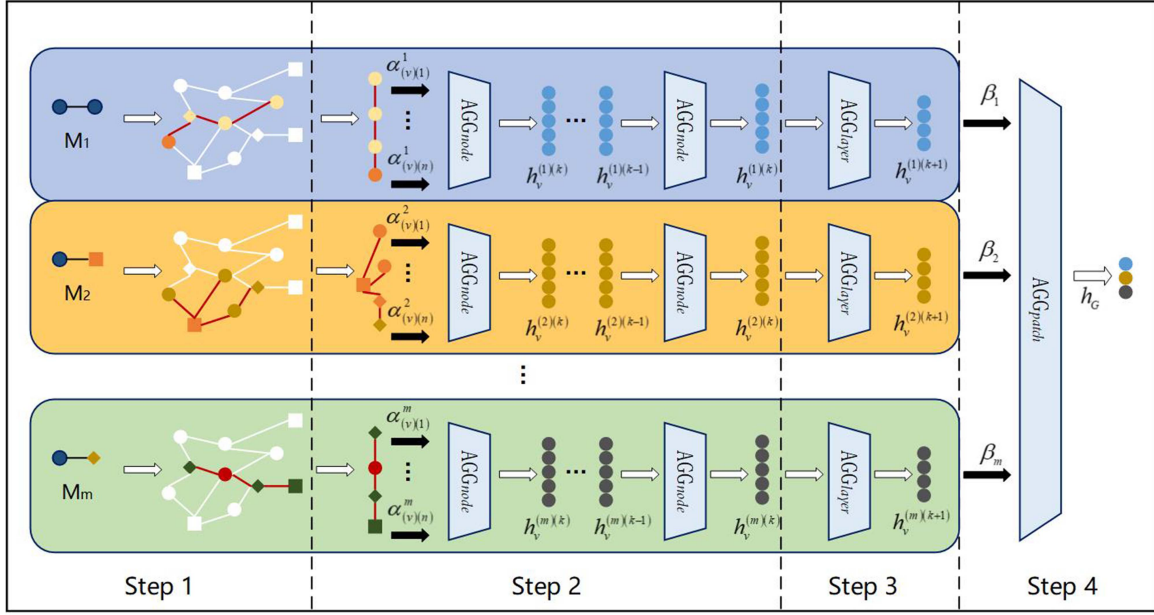


Fig. 5. HGAN structure consisting of four steps.

through four steps: meta-path sampling, node-level attention, hierarchical feature aggregation, and path-level attention to obtain the similarity score h_G of the two subgraph feature vectors. Inspired by [60], [61], [62] and [63], the structure of HGAN in LLM-HGAN including four steps: 1) meta-path sampling, where heterogeneous provenance data are converted into a unified graph representation; 2) node-level attention, LLM-HGAN utilizes neighbor node information from each subpath to learn the semantic information of the current node; 3) hierarchical feature aggregation, LLM-HGAN employs a hierarchical neural network architecture to capture the semantic and structural information of this meta-path using node-level attention networks; and 4) path-level attention, Feature vectors for each meta-path were generated through node-level attention networks and hierarchical dense connection networks, As shown in Fig. 5.

In heterogeneous graph attention networks, sensitivity is not a directly tunable parameter but is automatically learned through its hierarchical attention mechanism (node-level attention and semantic-level attention). Therefore, the core of adjusting “sensitivity” lies in influencing the distribution of attention weights through model design and the training process. LLM-HGAN learns node embedding vectors by considering context weight coefficients at three levels: node-level, layer-level, and path-level. As shown in Fig. 5, multiple meta-paths are first sampled using the heterogeneous graph context search algorithm (Fig. 5 Step 1). Then LLM-HGAN designed a node-level attention network to aggregate node features along the meta-path (Fig. 5 Step 2). Second, by integrating node features from different layers through layer-level dense-connected neural networks (Fig. 5 Step 3). Finally, a path-level attention network is designed to learn weight coefficients between different paths and generate a feature vector for the entire subgraph (Fig. 5 Step 4). The following content provides a further introduction to the computational process. For the input subgraph $SG = (V, E)$,

a meta-path set $M = \{M_1, M_2, \dots, M_m\}$ is generated through a context-based search process. Specifically, the process first randomly selects a meta-path schema $MS_i = A_1 \xrightarrow{R_1} A_2 \xrightarrow{R_2} \dots A_t \xrightarrow{R_t} A_{t+1} \dots \xrightarrow{R_{l-1}} A_l$, and then performs a random walk on the subgraph according to this schema.

The LLM-HGAN baseline is based on a hierarchical attention mechanism that includes node-level and semantic-level attention. Node-level attention assesses the significance of nodes relative to their neighbors in an isomorphic subgraph generated by each meta-path, dynamically assigning weights. Semantic-level attention fuses node representations from various meta-paths with importance weighting, enabling the model to learn each meta-path’s contribution to downstream tasks automatically. The transition probability from node v_i to node v_{i+1} at step i is defined in (3).

$$P(v_{i+1}|v_i, MS_i) = \begin{cases} \frac{1}{|N_{i+1}(v_i)|}; (v_i, v_{i+1}) \in E, \varphi(v_{i+1}) = A_{t+1}, \\ 0; (v_i, v_{i+1}) \in E, \varphi(v_{i+1}) \neq A_{t+1}, \\ 0; (v_i, v_{i+1}) \notin E. \end{cases} \quad (3)$$

Where $\varphi(v_i) = A_t$, $N_{i+1}(v_i)$ are the neighbour node of node v_i with node type A_{i+1} , meta-path sampling will repeat this process. Among them, the meta-path neighbours of node v in meta-path $M_i (1 \leq i \leq m)$ are defined as shown in (4):

$$N_v^i = \{u | (u, v) \in M_i, (v, u) \in M_i\}. \quad (4)$$

The importance of node u to node v along meta-path i is captured by e_{vu}^i , which is used to learn the attention weight coefficient α_{vu}^i . For a node pair (v, u) in the meta-path, this importance is computed as shown in (5).

$$e_{vu}^i = ATT_{node}(h_v, h_u; M_i). \quad (5)$$

Where ATT_{node} represents the neural network structure of the computational node-level attention. After obtaining the importance between node pairs based on the meta-path, this research normalize them using (6) to obtain the weight coefficient α_{vu}^i :

$$\alpha_{vu}^i = SoftMax(e_{vu}^{(i)(k)}) = \frac{\exp\left(\sigma\left(b\left[h_v^{(i)(k)} || h_u^{(i)(k)}\right]\right)\right)}{\sum_{j \in N_v^i} \exp\left(\sigma\left(b\left[h_v^{(i)(k)} || h_j^{(i)(k)}\right]\right)\right)}. \quad (6)$$

Then, the feature vector of node v on the meta-path is aggregated from the feature information of its neighbours and the relevant coefficients, as shown in (7):

$$h_v^{(i)(k)} = \sigma\left(\sum_{j \in N_v^i} \alpha_{vu}^{(i)(k-1)} \cdot h_v^{(i)(k-1)}\right). \quad (7)$$

Here, $k \in 1, 2, \dots, K$ represents the index of the network layer, $h_v^{(i)(k)}$ denotes the feature vector of process node v in meta-path M_i , and σ denotes the activation function. The feature aggregation process of the hierarchical neural network structure AGG_{layer} is shown in (8):

$$\begin{aligned} h_v^{(i)(k)} &= AGG_{layer}(h_v^{(0)}, h_v^{(1)}, \dots, h_v^{(k)}) \\ &= MLP([h_v^{(0)} : h_v^{(1)} : \dots : h_v^{(k)}]). \end{aligned} \quad (8)$$

Where $[]$ denotes the aggregation operation and $h_v^{(i)(k)}$ denotes the feature vector of process node v of meta-path M_i . At the same time, use (9) to calculate the weight coefficient of the eigenvector $h_v^{(i)(k+1)}$ of the meta-path M_i :

$$\beta_i = \frac{\exp\left(\sigma\left(b\left[W_b h_v^{(i)(k+1)} || W_b h_v^{(j)(k+1)}\right]\right)\right)}{\sum_{j \in m} \exp\left(\sigma\left(b\left[W_b h_v^{(i)(k+1)} || W_b h_v^{(j)(k+1)}\right]\right)\right)}. \quad (9)$$

Where $h_v^{(j)(k+1)}$ denotes the eigenvector of the subpath M_j , b denotes the trainable parameters, W_b denotes the weight coefficient matrix, $||$ denotes the concat operation, and σ denotes the nonlinear gate function. The feature aggregation of all meta-paths is calculated by (10):

$$h_G = AGG_{path} = \sum_{i=1}^m ATT(h_v^{(i)(k+1)}) h_v^{(i)(k+1)}. \quad (10)$$

Referencing the research methodologies in [43] and [45], and drawing from the proven effectiveness demonstrated in most literature and practice, the optimal choice is to set the attention layer depth to 2. The primary reason for this is the ‘‘over-smoothing’’ issue inherent to heterogeneous graphs, where networks with excessive layers tend to obscure the intrinsic semantic information of nodes and crucial meta-path information. During training, the dataset consists of p subgraph pairs (G_1^i, G_2^i) , $i \in \{1, 2, \dots, p\}$, and each subgraph pair has a corresponding label $y_i \in \{+1, -1\}$. The similarity score for each pair of graphs is calculated using the cosine similarity function, as shown in (11).

$$Sim(G_1^i, G_2^i) = \cos(h_{G_1^i}, h_{G_2^i}) = \frac{h_{G_1^i} \cdot h_{G_2^i}}{||h_{G_1^i}|| \cdot ||h_{G_2^i}||}. \quad (11)$$

In the above equation, Sim denotes similarity, $\cos$ denotes the cosine similarity calculation function, and $h_{G_1^i}$ and $h_{G_2^i}$ denote the feature vectors of the two process subgraphs. The loss function is calculated using (12).

$$l = \sum_{i=1}^p (Sim(G_1^i, G_2^i) - y_i)^2. \quad (12)$$

Refer to [64], [65] for the method of handling linear similarity between two sequences, the similarity score between subgraphs is calculated using (13). Among them, $h_{G_1^i}$ and $h_{G_2^i}$ represent the feature vectors of the two process subgraphs, and their similarity scores are positively correlated with the process semantic similarity. $h_{G_1^i}$ and $h_{G_2^i}$ learn process dynamics (e.g., context interaction, type) via a heterogeneous graph attention network, and cosine similarity comprehensively compares their multi-faceted similarities.

$$Sim(G_1^i, G_2^i) = \frac{\sum_{e_i \in h_{G_1^i}} \sum_{e_j \in h_{G_2^i}} e_i \cdot e_j}{\sqrt{\sum_{e_i \in h_{G_1^i}} e_i^2} \times \sqrt{\sum_{e_j \in h_{G_2^i}} e_j^2}}. \quad (13)$$

LLM-HGAN uses similarity learning to optimize inter-process distances between benign and malicious activities, enabling malicious process detection via feature vector analysis with provenance graphs. It identifies anomalies by comparing process vector similarity to predefined thresholds λ . This study finds that when λ is too small, all nodes are classified as normal nodes, while when λ is too large, all nodes are classified as malicious nodes. This prevents the correct classification of normal and malicious nodes, and the corresponding λ values differ across different datasets. In this study, λ denotes an empirical parameter, set to 0.63 for the ATLAS [34] dataset and 0.53 for the DARPA TC dataset.

C. TTP-Based Attack Detection Module (Stage II)

The output of the stage I is an abnormal process and serves as the input for the stage II, but it still needs to be processed through DFS to generate a suspected attack path before TTP can be used for attack comparison. LLM-HGAN employs clustering algorithms to group malicious processes executing coordinated attacks, then develops a temporally-aware forward DFS (Depth-First Search) approach to reconstruct attack paths by correlating distributed behaviors. It matches path behaviors with ATT&CK TTPs via designed mapping rules to detect APT events, and improves DFS to associate resource nodes (e.g., files) with processes (Algorithm 3).

The input of Algorithm 3 is the original provenance graph without preprocessing, because this graph retains all system entities and dependencies, and the set of abnormal process $V = \{v_1^p, v_2^p, \dots, v_s^p\}$ obtained in the Stage I (Section IV-B), and the output is the attack path. There may be multiple attack paths obtained through Algorithm 3. LLM-HGAN uses threat severity to determine the path most likely to contain a APT attack. Threat severity is calculated using (14):

$$f_u = \frac{1}{Freq(prov)} \sum_{e_i \in LLM-HGAN_{paths}} \frac{1}{Freq(e_i)}. \quad (14)$$

Algorithm 3: Attack path extraction.

```

1: Input: Original provenance graph  $G$ , abnormal process  $V$ 
2: Output:  $LLM - HGAN\_paths$ ;  $\triangleright$  Set of attack paths
3:  $path \leftarrow v_1^p$ 
4: For each  $v_i^p : V = \{v_2^p, \dots, v_s^p\}$  do
5:   For each  $path : LLM - HGAN\_paths$  do
6:     if  $havepath(v_{i-1}^p \in path, v_i^p)$  then
7:        $path \leftarrow path(v_{i-1}^p \in path, v_i^p)$ 
8:     else
9:        $newpath \leftarrow v_i^p$ 
10:       $PTHGL \leftarrow APT\_paths \leftarrow newpath$ 
11:    end if
12:  end for
13: end for
14: For each  $path : LLM - HGAN\_paths$  do
15:   For each  $v^p : LLM - HGAN\_paths$  do
16:      $path \leftarrow aDFS(G, v^p)$ 
17:   end for
18: end for

```

In the above equation, $Freq(prov)$ denotes the number of file processing events in the provenance graph $prov$, and $Freq(e_i)$ denotes the frequency of event e_i within the attack paths ($LLM-HGAN_paths$). A smaller $Freq(e_i)$ corresponds to a larger f_u , indicating that infrequently occurring file processing events are more likely to signify core cyber attack activities, whereas frequently observed events in provenance graphs typically reflect benign background tasks.

LLM-HGAN defines text generation TTP mapping rules M based on TTP technology, where $M = E \cup R$, E represents the set of conditions that system entities must satisfy, and R represents the set of conditions that causal events must satisfy. TTP mapping rules are a set of rules that include entity mapping rules and relationship mapping rules, APPENDIX A shows an example of mapping rules for attack technique TTP1566.

The TTP mapping rule matching algorithm leverages predefined TTP rules to associate attack paths with corresponding TTPs, thereby bridging the semantic gap between system logs and abstract adversarial behaviors (Algorithm 4). To streamline research, this study adopts an established combination of Dedupe, PyKEEN, and LLM Prompt to tackle entity and relation matching tasks. The algorithm takes TTP rules and attack paths as inputs and produces APT attack paths annotated with their matched TTPs as outputs. During matching, path events are compared against the rules, and an event is identified as an APT activity only when both the entity and relationship constraints are satisfied. The proposed LLM-HGAN's path-aware identification methodology significantly enhances the detection accuracy of multi-stage APT attack campaigns by enabling sequential phase analysis.

D. LLM-Based TTP Generation Module

The LLM-HGAN framework demonstrates that TTP-based detection efficacy critically depends on the temporal relevance

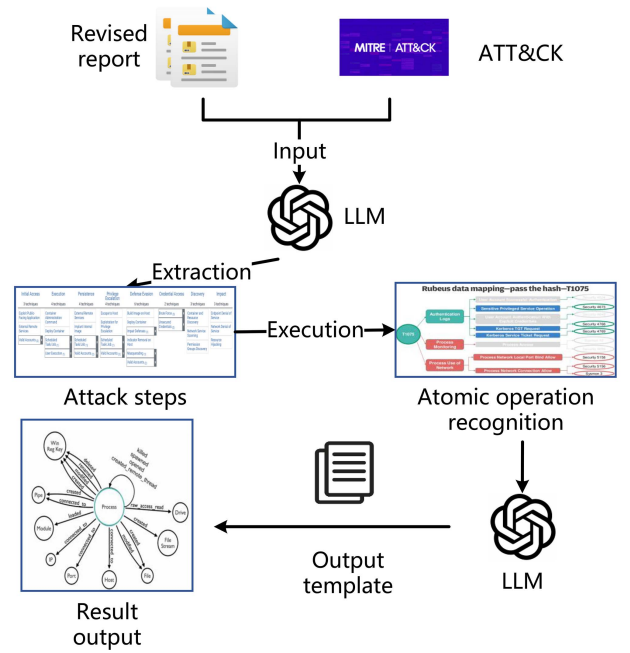


Fig. 6. The framework for generating ATT & CK TTPs using LLM.

and precision of indicators. Leveraging multi-task learning and contextual comprehension capabilities, LLMs enable systematic TTP generation for APTs by effectively integrating heterogeneous threat intelligence. Monica et al. (2022) introduced how to use LLM for zero-shot and few-shot information extraction in clinical texts, as well as how to use guided prompts and parsers to improve the structure of the output [66]. Wei et al. (2023) developed a powerful information extraction (IE) model by directly prompting an LLM, converting the zero-shot IE task into a multi-round QA problem [67]. Polak et al. (2024) proposed the ChatExtract method, which utilizes advanced conversational large LLM to perform highly accurate data extraction in a fully automated manner with minimal background knowledge [68]. Recent studies [69][70] show that as LLMs improve their ability to process long texts, background knowledge (e.g., network threat intelligence, output templates, enterprise ATT&CK frameworks) can be input. Given a prompt, LLM processes input and outputs it according to the template. Inspired by the above research findings, LLM-HGAN utilizes instruction prompts and contextual learning to generate APT attack TTPs by calling the LLM interface, as shown in Fig. 6.

The LLM-based TTP generation module identifies atomic operations based on attack steps from network threat intelligence, then outputs them according to the output template, thereby mapping the APT attack process into ATT&CK TTP.

Zhao et al. (2025) indicated that using large language models to address network architecture composition problems can lead to model output errors [71]. Issues like LLMs illusion and insufficient attack process details lead to accuracy and reliability problems in the generated ATT&CK TTP. This study identifies systemic challenges in deploying LLMs for TTPs, particularly persistent systemic data biases and model hallucination phenomena, which existing single-model approaches remain insufficient to address. The mitigation measure of LLM-HGAN is to propose

Algorithm 4: Based on TTP rule matching APT attack events.

```

1: Input: TTP Rules M; Attack path LLM-HGAN_paths
2: Output: Matching detected APT attack events
   Result_APT
3: Infopathlist ←
   GetPathList(LLM - HGAN_paths)
4: For each infopath : Infopathlist do
5:   isMatchedEntity ← MatchEntities(
     M.E, infopath)
6:   if isMatchedEntity is True then
7:     isMatchedRelation ← MatchRelation(
       M.R, infopath)
8:   end if
9:   if isMatchedRelation is True then
10:    Result_APT ←
      Result_APT ∪ (infopath, TTP)
11:   end if
12: end for

```

an ensemble learning framework based on model collision, which utilizes the complementary advantages of multiple heterogeneous LLMs and the majority voting mechanism to improve generation accuracy. Model collision refers to divergent outputs from multiple models given identical inputs during ensemble learning (15).

$$e_i = 1 - a_i, i \in \{1, 2, \dots, t\}, \quad (15)$$

In the above equation, t denotes the total number of LLM models. In the LLM-HGAN study, $t = 3$, corresponding to the use of three models: Qwen2.5-7B-Instruct, GPT-3.5 Turbo, and DeepSeek R1-0528-7B. The term a_i represents the accuracy of the i -th model ($0 \leq a_i \leq 1$), while e_i denotes its error rate. To evaluate the correlation between the error predictions of two models, Pearson's correlation coefficient ρ_{ij} is employed, with its specific formulation given in (16).

$$\rho_{ij} = \frac{\sum (e_{i,k} - \bar{e}_i)(e_{j,k} - \bar{e}_j)}{\sqrt{(\sum (e_{i,k} - \bar{e}_i)^2) \sum (e_{j,k} - \bar{e}_j)^2}}, \quad (16)$$

where $e_{i,k}$ and $e_{j,k}$ represent the error predictions of models i and j on sample k (error is 1, correct is 0), while $\bar{e}_i$ and $\bar{e}_j$ are calculated using (17):

$$\bar{e}_i = \frac{1}{n} \sum_{l=1}^n e_{i,l}, \quad \bar{e}_j = \frac{1}{n} \sum_{m=1}^n e_{j,m}. \quad (17)$$

For any two models i and j , their collision rate C_{ij} is estimated based on their error rates and the correlation of their error predictions, as shown in (18):

$$C_{ij} = e_i \times e_j \times \rho_{ij}, \quad (18)$$

where C_{ij} measures the probability that two models simultaneously give incorrect predictions. The overall collision rate can be calculated by considering all possible model combinations,

as shown in (19):

$$C = \frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^N C_{ij}. \quad (19)$$

Based on TTP conflicts among diverse large models, LLM-HGAN design an ensemble learning framework adopting majority voting. As a fundamental integration paradigm, its core principle is formally expressed by (20):

$$V(y) = \text{mode}\{\varepsilon_1 c_1(x), \varepsilon_2 c_2(x), \dots, \varepsilon_i c_i(x)\}. \quad (20)$$

In the above formula, $V(y)$ represents the final voting result, $c_i(x)$ represents the classification result of the i -th model for sample x ; ε_i is the weight coefficient used to control the proportion of outputs from LLMs of different scales; the $\text{mode}\{\}$ function returns the number that appears most frequently in a set of numbers, i.e., the category selected by the majority of models.

V. EXPERIMENT AND ANALYSIS

A. Research Questions and Experimental Objectives

This study systematically evaluates the performance, robustness, adaptability, and structural validity of LLMHGAN for APT attack detection, with emphasis on comparison against state-of-the-art approaches. Specifically, this research aim to address the following research questions:

- *RQ1: Comparative Detection Performance:* Can LLM-HGAN achieve higher accuracy and F1score than state-of-the-art provenancegraphbased APT detection methods?
- *RQ2: Detection of Unknown Vulnerability APTs:* Is LLM-HGAN capable of effectively identifying APT campaigns exploiting previously unseen vulnerabilities?
- *RQ3: Reliability of LLM Generated TTPs:* Can LLM generated TTPs maintain high accuracy and practical utility for APT detection in dynamic threat environments?
- *RQ4: Robustness to Family Variants and Evasion Techniques:* Does LLM-HGAN demonstrate resilience against familybased APT variants and sophisticated evasion strategies?
- *RQ5: Rationalization of LLM-HGAN using -order subgraph sampling operations:* Does the use of subgraphs to represent process behavior have an impact on anomaly detection?

B. Experimental Setup

1) *Hardware Configuration and Dataset Design for Comparative Experiments of Different Detection Methods:* The operational environment for implementing LLM-HGAN is described as follows. LLM-HGAN employs Python's NetworkX library to construct provenance graphs. The experimental environment is configured with an NVIDIA A100 GPU (128GB VRAM), an 8-core/16-thread CPU, and 1TB of storage. Development is carried out using Python 3.7 with PyTorch and TensorFlow 2.15.0, deployed on Ubuntu.

This study constructs the anomaly process detection dataset from ATLAS [34] and DARPA TC open-source datasets, the specific data details of ATLAS [34] are shown in Table III

TABLE III
DETAILS OF THE ATLAS [34] DATASET USED IN THE EXPERIMENT.

Attack ID	APT Campaign	Size(MB)	Entity	Event
S-1	Strategic web compromise	381	7,468	95.0K
S-2	Malvertising dominate	990	34,021	397.9K
S-3	Spam campaign	521	8,998	128.3K
S-4	Pony campaign	448	13,037	125.6K

below. While existing provenance-based APT detection systems predominantly utilize DARPA TC, its undisclosed evaluation audit logs necessitate supplementary data sources. LLM-HGAN incorporates ATLAS to address this gap, as both datasets are publicly accessible for direct download.

2) *Robustness Detection Experimental Environment for Familial APT and Unknown Attacks*: The experimental environments for family-based APT attack samples and unknown attack samples are configured as follows. The TTP-based APT detection experiments were conducted on an Ubuntu 22.04 LTS system equipped with an 8-core/16-thread CPU, 16GB RAM, and an NVIDIA RTX 2060 GPU. For unknown vulnerability detection, a Windows 10 (x64) virtual machine was configured with an Intel i9 CPU (1G), 128GB RAM, and a TITAN RTX GPU (24GB), on which APT simulation tools and diverse document samples were deployed. Family behavior analysis utilized the APTnotes, AlienVault OTX, and MalwareBazaar datasets (excluded from training). Notably, these datasets contain active APT29 and Lazarus samples [72].

3) *Generating TTP Using LLM Experimental Environment*: To fairly evaluate the accuracy of LLM-generated TTP strategies without affecting the detection capability of LLM-HGAN, this research configured a separate operational environment for the LLMs. LLM-based TTP generation requires collaborative computation across multiple models, necessitating high-performance system configurations. The experimental environment is deployed on an 8-GPU NVIDIA A100 cluster (HGX A100 motherboard) with Intel Xeon Scalable processors, 512GB DDR4 ECC memory, dual 480GB SSDs in RAID1, and 3.84TB NVMe SSD arrays connected through high-speed interconnects to enable full-bandwidth GPU communication.

The dataset consists of real-world APT threat intelligence collected from open-source platforms, including FreeBuf and OSINT. The LLMs employed in the experiments include Qwen2.5-7B-Instruct, GPT-3.5 Turbo (accessed via its API ecosystem with open-source toolchains), and DeepSeek R1-0528-7B.

C. Experimental Strategy

1) *Strategies for Comparative Testing of APT Detection Capabilities*: When designing experiments to evaluate the accuracy of APT attack detection technologies, [73], [74], [75], [76], [77] employed the method of using representative references as the comparison baseline. This study employs the ATLAS [34] and DARPA TC datasets' Engagement 3 project, specifically the CADETS and TRACE sub-datasets, as benchmarks for evaluating the LLM-HGAN model, comparing with state-of-the-art

baselines [34][78]. As provenance-graph detectors, ATLAS [34] and PROVIDECTOR [78] served as references. Performance is assessed via **Precision, Recall, and F1-score**.

2) *Strategies for Detecting of APT Attacks Exploiting Unknown Vulnerabilities*: This study assesses LLM-HGAN's detection of unknown APT activities via TTP methodologies. Real-world unknown APT samples are unavailable due to security, stakeholder, and privacy concerns. [79], [80]conducting similar experiments involves mixing conventional traffic with APT attack traffic, inspired by [24] [32] [33], this research utilizes network attack tools, as shown in APPENDIX B, to simulate multi-stage APT attacks in a controlled environment, focusing on complex data exfiltration and remote web penetration scenarios. To maximize proximity to a realistic cyberattack environment, the cyberattack tools are installed on other hosts within the experimental network environment. APT attack stages employ multiple software tools collaboratively, enabling tool rotation to simulate diverse attack patterns. The 217-day experiment evaluated detection performance using precision, recall, and F1-score, while assessing LLM-HGAN's efficacy against persistent APT campaigns. LLM-HGAN was benchmarked against the AirTag method [88] across diverse host environments to evaluate its detection capability for APT behaviors exploiting zero-day vulnerabilities.

3) *Strategies for Testing the Accuracy of TTPs Generated Based on LLM*: To ensure the accuracy of the experimental results evaluation, following the experimental design methodology outlined in [81], 10 high-quality APT attack threat intelligence reports were selected. Experimental validation utilized only 10 APT threat intelligence reports, constrained by legal restrictions on disclosing sensitive technical details of real APT incidents. Most publicly accessible threat intelligence reports lack sufficient detail or verifiable evidence, rendering them unsuitable for directly evaluating the accuracy of TTPs generated by LLMs. Future work will focus on mitigating the small-sample problem in threat intelligence via methods like transfer learning, alongside exploring strategies to defend against training data compromise. To ensure experimental operability and reproducibility while accounting for the practical utility of LLM-generated outputs, this study selected 10 high-quality cyber threat intelligence reports. 3 analysts with cyber security expertise conducted detailed, independent examinations of these reports and constructed TTPs along with annotated data based on a predefined ontology. The manually annotated results will subsequently be compared and analyzed against those generated automatically by the LLM-HGAN model. Determining an appropriate sample size of threat intelligence and assessing the feasibility of manual analysis efficiency remain key issues to be addressed in subsequent experimental stages. The experimental results will be analysed from the perspectives of **Precision, Recall, and F1-score**.

4) *Strategies for Testing the Ability to Detect APT Family Variants and Escapes*: The experiment used three mainstream and publicly available family-based APT attack datasets: APTnotes, AlienVault Open Threat Exchange (OTX), and MalwareBazaar. APTnotes includes 3 families and their 4 variants (2 families each have 1 variant, and the other family has 2 variants),

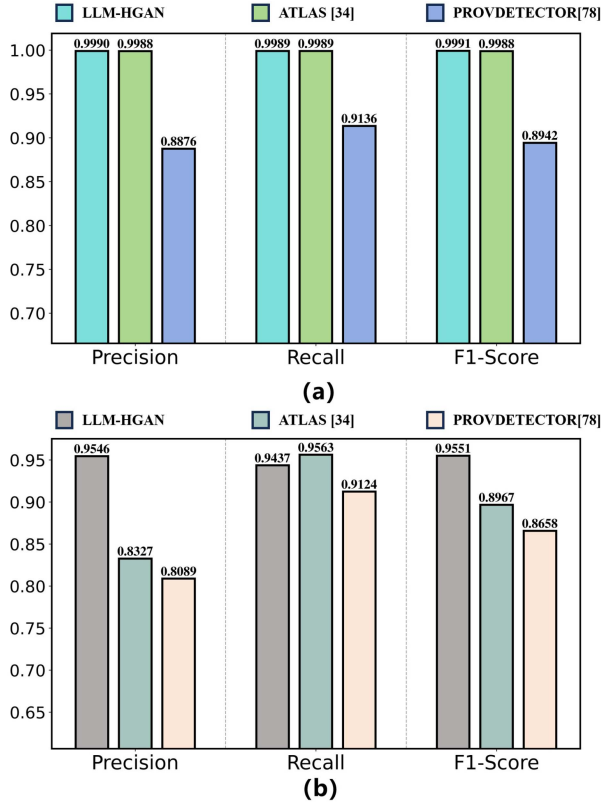


Fig. 7. Comparison of experimental results from LLM-HGAN, ATLAS [34], and PROVDETECTOR [78].

with variant patterns including base samples, obfuscated variants (packing/code obfuscation), and C&C protocol changes; OTX includes one family and its two variants, with variant patterns being base samples and obfuscated variants; MalwareBazaar includes one family and three variants, with variant patterns being base sample iterations and C&C protocol changes. The aforementioned datasets are not used in the model training of LLM-HGAN. Finally, this study analyses the experimental results in terms of variant detection rate, false alarm rate (FAR), and F1 score.

5) *Strategies for Testing the Rationality and Feasibility of K-Order Subgraph Sampling Operations:* During the experiments, refer to [82] for the approach to handling subgraphs, the ATLAS [34] dataset was sampled with 2nd-order subgraphs and the DARPA TC dataset with 6th-order subgraphs to capture process behavior. These representations were then compared against those derived from the full provenance graph, while keeping all other experimental parameters consistent.

D. Experimental Result and Analysis

1) *LLM-HGAN Significantly Outperforms Representative Methods of the Same Type in Terms of Accuracy in Detecting Abnormal Processes:* The experimental results are shown in Fig. 7, indicating that the comprehensive performance of LLM-HGAN is significantly superior to that of ATLAS [34] and PROVDETECTOR [78]. Specifically, the experiments conducted at the

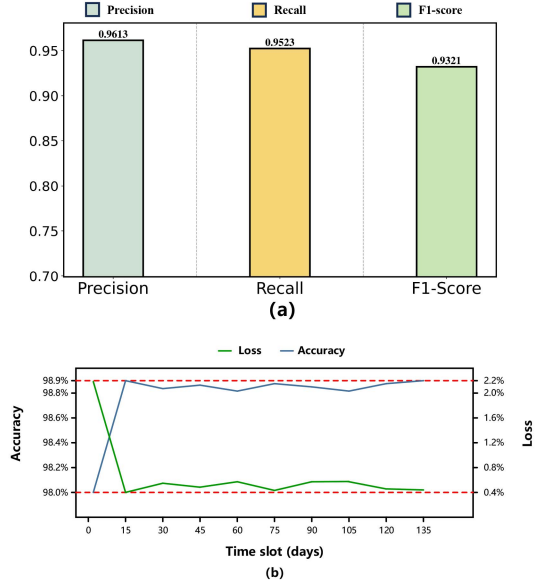


Fig. 8. Experimental results on the detection capability of LLM-HGAN for APT attacks based on unknown vulnerabilities.

ATLAS Lab dataset and the DARPA TC dataset are depicted in Fig. 7(a) and (b).

The reason for this result is that ATLAS [34] and PROVDETECTOR [78] use deep learning models based on natural language processing to classify the single execution path of a node, ignoring other adjacent nodes. In contrast, the proposed LLM-HGAN effectively models contextual dependencies among multi-step attack techniques while enhancing the discriminative separation between benign and malicious nodes via similarity-based metric learning. Concurrently, the integration of LLM with HGAN endows the LLM-HGAN detection model with outstanding scalability, thereby enabling robust performance across diverse datasets during experimentation.

2) *LLM-HGAN Has Excellent Capabilities for Detecting APT Attack Activities That Exploit Unknown Vulnerabilities:* The results in Fig. 8(a) present the evaluation of LLM-HGAN in terms of precision, recall, and F1-score. Based on extensive experimentation, the convergence behavior summarized in Fig. 8(b) shows that the detection accuracy stabilizes at approximately 98.9%, while the loss value converges to around 0.004. Meanwhile, in Fig. 8(b), the detection accuracy steadily increased over the first 15 days, stabilising thereafter. This precisely reflects the long-term self-learning and optimisation process of LLM-HGAN in countering unknown attack threats. Fig. 9 shows the comparative results of LLM-HGAN versus the AirTag method [88] and the HASGDetector method [46]. The x-axis denotes the count of distinct background applications (e.g., document editors, music players, games, programming development tools) installed on the target host (4, 6, 8, 10, 12, 14, 16), and the y-axis illustrates the accuracy rate distribution from 237 runs of both methods on the experimental host. Results indicate that LLM-HGAN outperforms AirTag in detection performance and generalization capability. While AirTag's accuracy improves

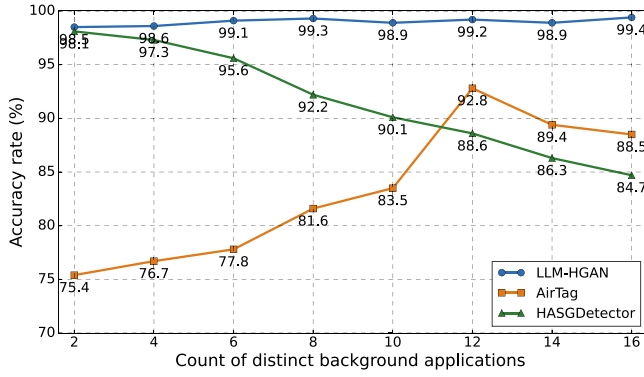


Fig. 9. Comparative experimental results under different system log background conditions. .

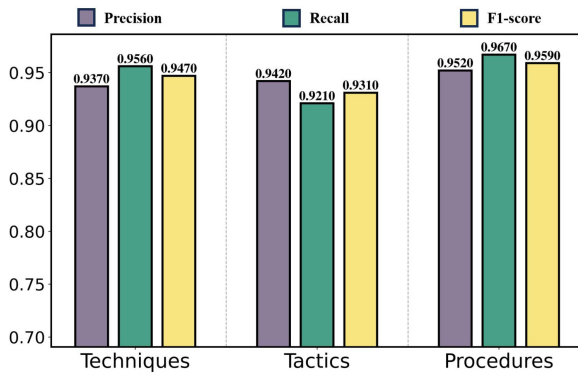


Fig. 10. The accuracy of LLM-HGAN in generating TTP.

with more iterations, its performance deteriorates as background software increases (generating more log files)—consistent with the limitation in [88] where its effectiveness depends on training data availability. As noted in [46], HASGDetector excels at learning normal behavior patterns and detecting unknown anomalies without prior knowledge or labeled data. While it achieves high accuracy in simple networks, its performance drops in complex environments. This is because, as network applications grow, it struggles to quickly learn diverse normal behaviors, leading to reduced detection accuracy.

These findings indicate that LLM-HGAN can reliably detect long-term APT attacks and continuously learn and adapt to evolving attack behaviors. Similar analysis approaches are also reported in [83], [84].

3) *LLM-HGAN Uses LLM to Generate TTPs With Outstanding Accuracy*: Fig. 10 presents the results, where the X-axis denotes the TTP types to be generated and the Y-axis shows the precision, recall, and F1-score. LLM-HGAN achieves high accuracy in TTP generation, reaching a practically usable level. The specific details of the experimental results regarding the accuracy of using LLM to generate TTP are shown in Table IV. Values in the table such as -1 and $+1$ represent the number of false negatives and false positives, respectively. The number of false negative and false positive cases in Table IV does not exceed 2, indicating that LLM-HGAN adopts ensemble

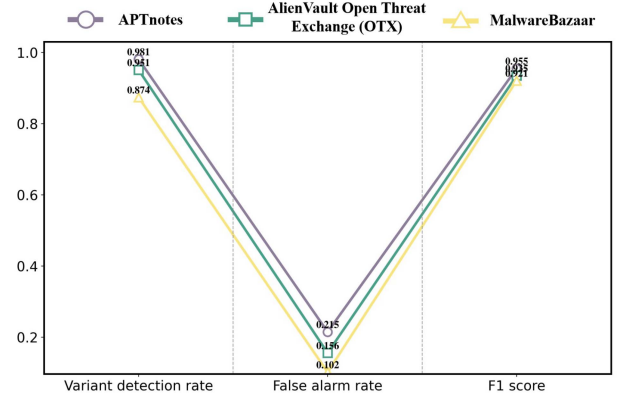


Fig. 11. Experimental results demonstrating the detection capability of LLM-HGAN for familial APT variants.

learning voting mechanism, which can effectively control the output error of LLMs. However, the precision of TTP generation remains strongly dependent on both the volume and fidelity of the training dataset.

4) *LLM-HGAN Demonstrates Remarkable Accuracy and Adaptability in Detecting APT Variants*: Drawing upon the analytical approach to experimental results outlined in [85], [86], [87], Fig. 11 shows that LLM-HGAN has excellent detection capabilities for variants of the APT family, with the highest recognition rate for variants of the APTnotes family. This is mainly because the open-source APTnotes dataset is comprehensive and rich, and the large model is used to continuously learn and update TTPs. The MalwareBazaar family's experimental results are insufficient due to the lack of comprehensiveness of the open source dataset. The fundamental reason lies in MalwareBazaar itself being a publicly accessible project. Constrained by legal considerations and attackers' reluctance to disclose details, the quantity and quality of authentic APT attack samples available are extremely limited. These samples are either relatively outdated versions, incomplete, or tampered attack chains. Furthermore, directly downloading samples into experimental environments may trigger minimal security policies (such as false positives for malicious activity), leading to unsatisfactory experimental results. Subsequent research may employ techniques like adversarial sample generation, transfer learning, and reinforcement learning to explore how to enhance the generalization capability of LLM-HGAN under small-sample conditions. Experimental results also indicate that LLM-HGAN has achieved practical detection levels for familial APT variants.

5) *LLM-HGAN Significantly Improves Anomaly Detection Using Subgraph Representation of Process Behavior*: The experimental results are shown in Fig. 12, the x-axis represents the percentage and the y-axis represents the effect of comparing the use of the entire provenance graph and k-order subgraph sampling from three perspectives, Fig. 12(a)–(b) present experimental results using the ATLAS [16] and DARPA TC datasets, respectively.

TABLE IV
RESULTS ON THE ACCURACY OF LLM-HGAN IN GENERATING TTP.

APT attack threat intelligence reports	Tactics identification		Techniques identification		Procedures identification	
	Annotated data	LLM-HGAN	Annotated data	LLM-HGAN	Annotated data	LLM-HGAN
1	6	-0(+0)	6	-0(+0)	5	-0(+0)
2	5	-1(+1)	6	-0(+1)	6	-0(+0)
3	5	-0(+1)	6	-1(+0)	4	-0(+0)
4	4	-0(+1)	5	-0(+2)	6	-0(+1)
5	6	-0(+1)	6	-1(+0)	4	-1(+2)
6	6	-1(+0)	7	-2(+1)	5	-0(+1)
7	3	-0(+2)	4	-0(+1)	7	-1(+0)
8	6	-1(+0)	6	-2(+1)	5	-0(+1)
9	4	-0(+1)	7	-0(+1)	3	-1(+0)
10	6	-0(+0)	4	-0(+2)	4	-0(+2)
Precision	1.00	0.937	1.00	0.942	1.00	0.952
Recall	1.00	0.956	1.00	0.921	1.00	0.967
F1-score	1.00	0.947	1.00	0.931	1.00	0.959

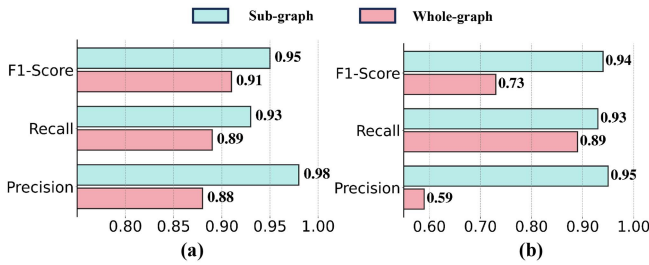


Fig. 12. Experiments on the operational rationality of LLM-HGAN sampling using k-order subgraphs.

Full provenance graphs hinder process detection by trapping irrelevant host activities. When learning node features, redundant data interference blurs key patterns.

E. Ablation Experiment

To rigorously validate the seamless integration of LLM and HGAN within the proposed LLM-HGAN framework, as well as to quantitatively demonstrate the substantial performance improvements achieved by this synergistic fusion in the context of APT detection efficacy, detection accuracy served as the primary metric. To evaluate the performance of the four methods, this experiment conducted ablation experiments on 100 distinct APT attack process samples, each generated using cyberattack tools randomly selected from Appendix B. For each method, three independent trials were conducted, and the final results were obtained by averaging across the three runs. Similar evaluation protocols have been adopted in prior studies [24], [51], [73]. The ablation experiment results are shown in Table V, wherein the GCN were implemented using PyTorch Geometric. Experimental results show that the LLM-HGAN architecture achieves the highest APT detection accuracy, while

LLM+GCN yields the lowest. The observed performance gap originates from the uniform weighting of neighboring nodes in GCN-based methods, which persists even when LLMs enhance TTP generation. This aggregation scheme limits efficiency and accuracy in complex, heterogeneous provenance graphs. Conversely, HGAN employs attention to dynamically weight key nodes, offering a refined and more effective alternative to standard GCN.

VI. CONCLUSIONS AND FUTURE WORK

This study presents LLM-HGAN, a novel framework that integrates HGAN with LLM capabilities to detect APTs through TTP analysis. Existing detection approaches commonly rely on attribution graphs, yet they suffer from two notable limitations: i) rule-based methods often generate a large number of false positives for benign system activities, and ii) provenance graphs constructed from system logs are typically very large, making it difficult to efficiently locate malicious behaviors. To address these challenges, LLM-HGAN adopts a two-step detection paradigm. First, it combines similarity learning with a heterogeneous graph attention network to improve the detection rate of abnormal processes. Next, the framework applies anomaly process clustering and an enhanced forward traversal algorithm to identify potential attack paths, which are subsequently matched against TTP rules for attack detection. In addition, the system leverages LLMs to enable automated and continuous updating of the TTP rule base. Empirical evaluations demonstrate that LLM-HGAN effectively withstands detection evasion techniques, complex attack patterns, and prolonged covert activities across four experimental scenarios, establishing itself as a highly promising innovative approach. The computational demands of the LLM-HGAN approach can be effectively managed through techniques including graph sampling, meta-path optimization, model lightweighting, and feedback-based optimization, enabling its application to

TABLE V
RESULTS OF ABLATION EXPERIMENT.

LLM+HGAN	HGAN	LLM	LLM+GCN	Detection accuracy 1	Detection accuracy 2	Detection accuracy 3	Average detection accuracy
✓	×	×	×	94.5%	95.2%	96.7%	95.5%
×	✓	×	×	83.2%	80.7%	81.5%	81.8%
×	×	✓	×	62.5%	61.7%	61.6%	61.8%
×	×	×	✓	27.5%	26.3%	27.4%	27.1%

TABLE VI
TTP1566 RULE: PHISHING TECHNIQUES.

TTP rules	Project	Name	Rules
Entity mapping rules	S	Socket	$s == \text{socket} \wedge s.addr_ip \notin \{public_ip\}$
	P	Process	$p.type == process \wedge p.ppid == p.pid$
Relationship mapping rules	$execute(p, f_1)$	File execution	$p.cmd_line = f.path$
	$connect(p, s)$	Network connection	$p.domain == s.url$
	$recv(s, f_2)$	Data readout	$p.local_addr == s.remote_addr$

TABLE VII
TOOLS USED IN SIMULATED UNKNOWN APT ATTACK DETECTION EXPERIMENTS.

APT attack phase	Cyber security tools	Description
Reconnaissance	ARL, OWASP, Altdns	Target reconnaissance and asset Detection
Weapon construction	Metasploit	Customised malicious payloads
Payload delivery	SET, Gophish, Veil-Evasion	Attack payload delivery phase
Vulnerability exploitation	Nessus, Metasploit, Nikto	Convert malicious payloads into actual code execution
Implant installation	Sliver, Empire	Deploy persistent malicious programs
C&C	Nc, Metasploit	Remote control channel
Task execution	Rclone, CloakQuest	Carry out scheduled attack activities

small-to-medium-scale heterogeneous graphs. To address computational cost and operational feasibility, open-source LLMs can be utilized with strategies including selective parameter updates, lightweight models, and periodic retraining.

The current implementation of TTP rule derivation remains limited to English-language LLM-based approaches. Future investigations will explore: i) integration of multilingual APT threat intelligence feeds, and ii) integration of multilingual APT threat intelligence sources and continuous collection of high-value threat intelligence. iii) conduct research into adversarial sample generation, producing diverse APT attack samples to continually refine the generalization capabilities of LLM-HGAN.

APPENDIX A

An example of mapping rules for TTP1566 is shown in Table VI.

APPENDIX B

Tools of detecting unknown APT are shown in Table VII.

REFERENCES

- [1] S. Feng, Z. Xiong, D. Niyato, and P. Wang, "Dynamic resource management to defend against advanced persistent threats in fog computing: A game theoretic approach," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 995–1007, Jul./Sep. 2021, doi: [10.1109/TCC.2019.2896632](#).
- [2] S. Feng, Z. Xiong, D. Niyato, P. Wang, and A. Leshem, "Evolving risk management against advanced persistent threats in fog computing," in *Proc. IEEE 7th Int. Conf. Cloud Netw.*, Tokyo, Japan, Oct. 2018, pp. 1–6, doi: [10.1109/CloudNet.2018.8549403](#).
- [3] L.-X. Yang, K. Huang, X. Yang, Y. Zhang, Y. Xiang, and Y. Y. Tang, "Defense against advanced persistent threat through data backup and recovery," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 3, pp. 2001–2013, Jul./Sep. 2020, doi: [10.1109/TNSE.2020.3040247](#).
- [4] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Commun. Surv. Tut.*, vol. 21, no. 2, pp. 1851–1877, Second quarter 2019, doi: [10.1109/COMST.2019.2891891](#).
- [5] Y. Wang, H. Liu, Z. Li, Z. Su, and J. Li, "Combating advanced persistent threats: Challenges and solutions," *IEEE Netw.*, vol. 38, no. 6, pp. 324–333, Nov. 2024, doi: [10.1109/MNET.2024.3389734](#).
- [6] J. Liang, S. Guo, Z. Hong, E. Zhou, C. Zhang, and B. Xiao, "SecPQ: Secure prediction queries on encrypted outsourced databases," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 4534–4548, Sep./Oct. 2025, doi: [10.1109/TDSC.2025.3549052](#).
- [7] Z. Peng et al., "Generative artificial intelligence models for emerging communication systems: Fundamentals and challenges," *IEEE Commun. Mag.*, vol. 63, no. 9, pp. 36–43, Sep. 2025, doi: [10.1109/MCOM.001.2400730](#).
- [8] H. Yue, T. Li, D. Wu, R. Zhang, and Z. Yang, "Detecting APT attacks using an attack intent-driven and sequence-based learning approach," *Comput. Secur.*, vol. 140, 2024, Art. no. 103748, doi: [10.1016/j.cose.2024.103748](#).
- [9] M. L. Martin, "Cyber kill chain," 2022. [Online]. Available: <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>
- [10] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *Proc. 2020 IEEE Symp. Secur. Privacy*, May 2020, pp. 1172–1189, doi: [10.1109/SP40000.2020.00096](#).
- [11] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," in *Proc. 27th Annu. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2020, pp. 1–6, doi: [10.14722/ndss.2020.24046](#).
- [12] Y. Hu, Y. Liu, Z. Lv, Y. Chen, Z. Lin, and Y. Du, "Privacy-preserving few-shot traffic detection against advanced persistent threats via federated meta learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2549–2560, May/Jun. 2023, doi: [10.1109/TNSE.2023.3304556](#).
- [13] H. Li, Y. Liu, Z. Lv, Y. Chen, Z. Lin, and Y. Du, "Explainable intelligence-driven defense mechanism against advanced persistent threats: A joint edge game and AI approach," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 757–775, Mar./Apr. 2021, doi: [10.1109/TDSC.2021.3130944](#).

- [14] X. Yang, G. Li, J. Wu, K. Zhou, J. Li, and W. Yang, "Backdoor-empowered regulable privilege authorization for edge-level graph learning in 6G vehicular networks," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 6307–6318, May 2025, doi: [10.1109/TCE.2025.3533648](https://doi.org/10.1109/TCE.2025.3533648).
- [15] W. Huang, G. Li, M. Chen, J. Li, and H. Zhu, "Silent penetrator: Breaching cross-domain federated fine-tuning via feature shift-induced backdoor," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 7106–7120, 2025, doi: [10.1109/TIFS.2025.3581026](https://doi.org/10.1109/TIFS.2025.3581026).
- [16] R. Xu et al., "Toward covert and reliable communication for anti-eavesdropping transmission in V2X networks," in *IEEE Trans. Wireless Commun.*, vol. 24, no. 8, pp. 6429–6442, Aug. 2025, doi: [10.1109/TWC.2025.3553132](https://doi.org/10.1109/TWC.2025.3553132).
- [17] J. Chen, X. Lan, Q. Zhang, W. Ma, W. Fang, and J. He, "Defending against APT attacks in cloud computing environments using grouped multiagent deep reinforcement learning," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 19459–19470, Jun. 2025, doi: [10.1109/JIOT.2025.3542119](https://doi.org/10.1109/JIOT.2025.3542119).
- [18] L. Xiao, H. Liu, Z. Lv, Y. Chen, Z. Lin, and Y. Du, "Reinforcement-learning-based APT defense for large-scale smart grids," *IEEE Internet Things J.*, vol. 12, no. 9, pp. 11917–11925, May 2025, doi: [10.1109/JIOT.2024.3519134](https://doi.org/10.1109/JIOT.2024.3519134).
- [19] W. Cheng et al., "TAGAPT: Toward automatic generation of APT samples with provenance-level granularity," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 4137–4151, 2025, doi: [10.1109/TIFS.2025.3557742](https://doi.org/10.1109/TIFS.2025.3557742).
- [20] S. Moothedath et al., "Dynamic information flow tracking for detection of advanced persistent threats: A stochastic game approach," *IEEE Trans. Autom. Control*, vol. 69, no. 10, pp. 6684–6699, Oct. 2024.
- [21] M. M. Anjum, S. Iqbal, and B. Hamelin, "ANUBIS: A provenance graph-based framework for advanced persistent threat detection," in *Proc. 37th ACM/SIGAPP Symp. Appl. Comput.*, Apr. 2022, pp. 1684–1693, doi: [10.1145/3477314.3507097](https://doi.org/10.1145/3477314.3507097).
- [22] A. Ding, G. Li, X. Yi, X. Lin, J. Li, and C. Zhang, "Generative AI for software security analysis: Fundamentals, applications, and Challenges," *IEEE Softw.*, vol. 41, no. 6, pp. 46–54, Nov./Dec. 2024, doi: [10.1109/MS.2024.3416036](https://doi.org/10.1109/MS.2024.3416036).
- [23] K. Lan, G. Li, W. Huang, and J. Li, "HFL-RD: Heterogeneous federated learning-empowered ransomware detection via APIs and traffic features," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 5, pp. 4096–4111, Oct. 2025, doi: [10.1109/TNSM.2025.3574716](https://doi.org/10.1109/TNSM.2025.3574716).
- [24] K. A. Akbar et al., "Advanced persistent threat detection using data provenance and metric learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3957–3969, Sep./Oct. 2023.
- [25] A. Aly, S. Iqbal, A. Youssef, and E. Mansour, "MEGR-APT: A memory-efficient APT hunting system based on attack representation learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5257–5271, 2024.
- [26] T. Chen et al., "APT-KGL: An intelligent APT detection system based on threat knowledge and heterogeneous provenance graph learning," *IEEE Trans. Dependable Secure Comput.*, early access, Dec. 26, 2022, doi: [10.1109/TDSC.2022.3229472](https://doi.org/10.1109/TDSC.2022.3229472).
- [27] S. Wang et al., "THREATTRACE: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3972–3987, 2022.
- [28] H. Irshad et al., "TRACE: Enterprise-wide provenance tracking for real-time APT detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4363–4376, 2021.
- [29] T. Li, X. Liu, W. Qiao, X. Zhu, Y. Shen, and J. Ma, "T-Trace: Constructing the APTs provenance graph through multiple syslogs correlation," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1179–1195, May/Jun. 2024.
- [30] Y. Wu et al., "Paradise: Real-time, generalized, and distributed provenance-based intrusion detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1624–1640, Mar./Apr. 2023.
- [31] K. Kurniawan, A. Ekelhart, E. Kiesling, and G. Quirchmayr, "CRYSTAL: Knowledge graph-based framework for tactical attack discovery in audit data," *Comput. Secur.*, vol. 121, 2022, Art. no. 102828.
- [32] C. Xiong et al., "Conan: A practical real-time APT detection system with high accuracy and efficiency," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 551–565, Jan./Feb. 2022.
- [33] T. Chen et al., "APTSHIELD: A stable, efficient and real-time APT detection system for linux hosts," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5247–5264, Nov./Dec. 2023.
- [34] A. Alsaheel et al., "ATLAS: A sequence-based learning approach for attack investigation," in *Proc. 30th USENIX Secur. Symp.*, Berkeley, CA, USA, 2021, pp. 3005–3022.
- [35] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "PROGRAPHER: An anomaly detection system based on provenance graph embedding," in *Proc. 32nd USENIX Secur. Symp.*, Berkeley, CA, USA, 2023, pp. 4355–4372.
- [36] Z. Xu, P. Fang, C. Liu, X. Xiao, Y. Wen, and D. Meng, "DEPCOMM: Graph summarization on system audit logs for attack investigation," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2022, pp. 540–557, doi: [10.1109/SP46214.2022.9833632](https://doi.org/10.1109/SP46214.2022.9833632).
- [37] K. Akbar et al., "Advanced persistent threat detection using data provenance and metric learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3957–3969, Sep./Oct. 2023.
- [38] L. Zhou, J. Li, Z. Gu, J. Qiu, B. B. Gupta, and Z. Tian, "PANNER: POS-aware nested named entity recognition through heterogeneous graph neural network," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 4, pp. 4718–4726, Aug. 2024, doi: [10.1109/TCSS.2022.3159366](https://doi.org/10.1109/TCSS.2022.3159366).
- [39] Z. Li, X. Cheng, L. Sun, J. Zhang, and B. Chen, "A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks," *Secur. Commun. Netw.*, vol. 9, 2021, Art. no. 5588062.
- [40] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 699–714, 2023, doi: [10.1109/TIFS.2022.3228493](https://doi.org/10.1109/TIFS.2022.3228493).
- [41] C. Liu, B. Li, J. Zhao, Z. Zhen, X. Liu, and Q. Zhang, "FewM-HGCL: Few-shot malware variants detection via heterogeneous graph contrastive learning," *IEEE Trans. Dependable Secure Comput.*, early access, Oct. 25, 2022, doi: [10.1109/TDSC.2022.3216902](https://doi.org/10.1109/TDSC.2022.3216902).
- [42] L. Zhang, "Implementing RGCN model in network security Big Data analysis," *J. Cyber Secur. Mobility*, vol. 14, no. 2, pp. 505–530, 2025, doi: [10.13052/jcsm2245-1439.14210](https://doi.org/10.13052/jcsm2245-1439.14210).
- [43] L. Wu et al., "Efficient intrusion detection via heterogeneous graph attention networks and parallel provenance analysis," *Comput. Netw.*, 270, 2025, Art. no. 111552.
- [44] H. Liu, C. Zeng, Z. Li, L. Lu, J. Chen, and Z. Zhou, "DAMAGE: Directed heterogeneous network attack sequence inference through graph attention matrix generation embedding and reinforcement learning," *IEEE Syst. J.*, vol. 19, no. 2, pp. 392–403, Jun. 2025.
- [45] Y. Hei et al., "HAWK: Rapid android malware detection through heterogeneous graph attention networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4703–4717, Apr. 2024.
- [46] G. Xie, X. Xu, H. Gao, and M. Iqbal, "HASGDetect: An effective host-based Intrusion Anomaly detection framework with Large-Scale Attribute heterogeneous graphs," *IEEE Trans. Consum. Electron.*, vol. 71, no. 3, pp. 7523–7538, Aug. 2025, doi: [10.1109/TCE.2025.3592903](https://doi.org/10.1109/TCE.2025.3592903).
- [47] S. Benabderrahmane, P. Valtchev, J. Cheney, and T. Rahwan, "APT-LLM: Embedding-based anomaly detection of cyber advanced persistent threats using large language models," in *Proc. 13th Int. Symp. Digit. Forensics Secur.*, Boston, MA, USA, 2025, pp. 1–6, doi: [10.1109/ISDFS65363.2025.11011912](https://doi.org/10.1109/ISDFS65363.2025.11011912).
- [48] H. Xu, C. Si, Z. Zhou, C. Wang, P. Sun, and Q. Liu, "APTSniffer: Detecting APT attack traffic using retrieval-augmented large language models," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, Hyderabad, India, 2025, pp. 1–5, doi: [10.1109/ICASSP49660.2025.10888022](https://doi.org/10.1109/ICASSP49660.2025.10888022).
- [49] A. Selim, J. Zhao, and B. Yang, "Large language model for smart inverter cyber-attack detection via textual analysis of volt/VAR commands," *IEEE Trans. Smart Grid*, vol. 15, no. 6, pp. 6179–6182, Nov. 2024.
- [50] C. He, Y. Wang, J. Hu, T. H. Luan, Y. Bi, and Z. Su, "Collaborative vehicular threat sharing: A long-term contract-based incentive mechanism with privacy preservation," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 12, pp. 21528–21544, Dec. 2024.
- [51] T. Yu, G. Liu, C. Wang, and Y. Yang, "LLMGRAPH: Label-free detection against APTs in edge networks via LLM and GCN," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 7256–7271, Nov./Dec. 2025, doi: [10.1109/TDSC.2025.3596092](https://doi.org/10.1109/TDSC.2025.3596092).
- [52] M. Golec, Y. Khamayseh, S. Melhem, and A. Alwarafy, "LLM-driven APT detection for 6G wireless networks: A systematic review and taxonomy," *IEEE Access*, vol. 13, pp. 145271–145288, 2025.
- [53] Y. Hmimou, M. Tabaa, A. Khia, and Z. Hidila, "A multi-agent system for cybersecurity threat detection and correlation using large language models," *IEEE Access*, vol. 13, pp. 150199–150215, 2025.
- [54] F. Zuo, J. Rhee, and Y. Choe, "Knowledge transfer from LLMs to provenance analysis: A semantic-augmented method for APT detection," 2025, *arXiv:2503.18316*.
- [55] G. Leopoldo, D. Juan-Jose, S. Joerg, and M. Inmaculada, "LLM-Based cyberattack detection using network flow statistics," *Appl. Sci.-Basel*, vol. 15, no. 12, pp. 12–25, 2025, doi: [10.3390/app15126529](https://doi.org/10.3390/app15126529).

- [56] N. I. CheMat, N. Jamil, Y. Yusoff, and M. L. M. Kiah, "A systematic literature review on advanced persistent threat behaviors and its detection strategy," *J. Cybersecurity*, vol. 10, no. 1, 2024, Art. no. tyad023.
- [57] R. Li, X. Meng, and Y. Zhang, "Group-aware dynamic graph representation learning for next POI recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 5, pp. 2614–2625, May 2025, doi: [10.1109/TKDE.2025.3538005](https://doi.org/10.1109/TKDE.2025.3538005).
- [58] T. Zhu, J. Wang, L. Ruan, C. Xiong, J. Yu, and Y. Li, "General, efficient, and real-time data compaction strategy for APT forensic analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3312–3325, 2021, doi: [10.1109/TIFS.2021.3076288](https://doi.org/10.1109/TIFS.2021.3076288).
- [59] Y. Zhang, F. Zhang, H. Li, S. Zhang, X. Guo, and Y. Chen, "Data-aware adaptive compression for stream processing," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4531–4549, Sep. 2024, doi: [10.1109/TKDE.2024.3377710](https://doi.org/10.1109/TKDE.2024.3377710).
- [60] G. Mei, Z. Guo, L. Pan, Q. Li, F. Li, and S. Liu, "LIHAN: A lattice-guided incomplete heterogeneous information network embedding model for node classification," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 6, pp. 7411–7420, Dec. 2024, doi: [10.1109/TCSS.2024.3405569](https://doi.org/10.1109/TCSS.2024.3405569).
- [61] S. Fan, G. Liu, and J. Li, "A heterogeneous graph neural network with attribute emcompassion and structure-aware attention," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 1, pp. 829–838, Feb. 2024, doi: [10.1109/TCSS.2023.3239034](https://doi.org/10.1109/TCSS.2023.3239034).
- [62] F. Wu et al., "Multi-variate time series prediction of traffic and users for dynamic RRR-BBU mapping in C-RAN," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10557–10572, Oct. 2025, doi: [10.1109/TMC.2025.3570851](https://doi.org/10.1109/TMC.2025.3570851).
- [63] W. Wang et al., "HGATE: Heterogeneous graph attention auto-encoders," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3938–3951, Apr. 2023, doi: [10.1109/TKDE.2021.3138788](https://doi.org/10.1109/TKDE.2021.3138788).
- [64] J. He et al., "Advancing non-intrusive load monitoring: Predicting appliance-level power consumption with indirect supervision," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2957–2973, Jul./Aug. 2025.
- [65] W. Li et al., "MUCVR: Edge computing-enabled high-quality multi-user collaboration for interactive MVR," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 10, pp. 2058–2072, Oct. 2025, doi: [10.1109/TPDS.2025.3595801](https://doi.org/10.1109/TPDS.2025.3595801).
- [66] M. Agrawal, S. Hegselmann, H. Lang, Y. Kim, and D. Sontag, "Large language models are few-shot clinical information extractors," in *Proc. Conf. Empirical Methods Nat. Lang. Process.*, 2022, pp. 1998–2022.
- [67] X. Wei et al., "ChatIE: Zero-shot information extraction via chatting with ChatGPT," 2023, *arXiv:2302.10205v2*.
- [68] M. P. Polak and D. Morgan, "Extracting accurate materials data from research papers with conversational language models and prompt engineering," *Nature Commun.*, vol. 15, 2024, Art. no. 1569, doi: [10.1038/s41467-024-45914-8](https://doi.org/10.1038/s41467-024-45914-8).
- [69] J. Li, M. Wang, Z. Zheng, and M. Zhang, "LooGLE: Can long-context language models understand long contexts?," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 16304–16333.
- [70] Z. Dong, T. Tang, J. Li, W.X. Zhao, and J. Wen, "BAMBOO: A comprehensive benchmark for evaluating long text modeling capacities of large language models," in *Proc. Joint Int. Conf. Comput. Linguistics, Lang. Resour. Eval. (LREC-COLING)*, 2024, pp. 2086–2099.
- [71] J. Zhao, T. Wen, and K. Cheong, "Can large language models be trusted as evolutionary optimizers for network-structured combinatorial problems?," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 1191–1206, 2021.
- [72] H. M. Soliman, D. Sovilj, G. Salmon, M. Rao, and N. Mayya, "RANK: AI-assisted end-to-end architecture for detecting persistent attacks in enterprise networks," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3834–3850, Jul./Aug. 2024.
- [73] J. Mi, Q. Li, Z. Han, W. Liao, and J. Fu, "Graph learning on instruction stream-augmented CFG for malware variant detection," *IEEE Trans. Inform. Forensics Secur.*, vol. 20, pp. 3015–3030, 2025.
- [74] X. Li, X. Jiang, H. Wan, and X. Zhao, "TeRed: Normal behavior-based efficient provenance graph reduction for large-scale attack forensics," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 9463–9476, 2025.
- [75] T. Yu, G. Liu, C. Wang, and Y. Yang, "A risk management approach to defending against the advanced persistent threat," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 6, pp. 1163–1172, Nov./Dec. 2020.
- [76] J. Yang, Q. Zhang, X. Jiang, S. Chen, and F. Yang, "POIROT: Causal correlation aided SemanticAnalysis for advanced PersistentThreat detection," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 6, pp. 3546–3563, Sep./Oct. 2020.
- [77] T. Li, Y. Jiang, C. Lin, S. Obaidat, Y. Shen, and J. Ma, "DeepAG: Attack graph construction and threats prediction with bi-directional deep learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 740–757, Jan./Feb. 2023.
- [78] Q. Wang, W. U. Hassan, D. Li, K. Jee, and X. Yu, "You are what you do: Hunting stealthy malware via data provenance analysis," in *Proc. Netw. Distrib. System Secur. Symp.*, San Diego, CA, USA, 2020, pp. 1–16.
- [79] Y. Hu, J. Wu, G. Li, J. Li, and J. Cheng, "Privacy-preserving few-shot traffic detection against advanced persistent threats via federated meta learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2549–2560, May/Jun. 2024.
- [80] C. Chen et al., "The last mile of attack investigation: Audit log analysis toward software vulnerability location," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 9566–9581, 2024.
- [81] X. He, H. Huang, C. Wang, F. Hu, T. Cai, and Z. Zheng, "A fairness-guaranteed framework for semi-asynchronous federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 6, pp. 4462–4479, Nov./Dec. 2025, doi: [10.1109/TNSE.2025.3572223](https://doi.org/10.1109/TNSE.2025.3572223).
- [82] B. Wang, B. Jiang, and C. Ding, "FL-GNNs: Robust network representation via feature learning guided graph neural networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 750–760, Jan./Feb. 2024.
- [83] G. Shenderovitz and N. Nissim, "Bon-APT: Detection, attribution, and explainability of APT malware using temporal segmentation of API calls," *Comput. Secur.*, vol. 142, 2024, Art. no. 103862.
- [84] Y. Shi, G. Li, J. Wu, J. Li, and H. Fang, "BPF-DAG: Byte-packet-flow features fusion via dynamic attributed graph for reliable encrypted traffic classification," *IEEE Trans. Inf. Forensics Secur.*, vol. 21, pp. 197–211, 2026.
- [85] X. Tang et al., "ROBY: A byzantine-robust and privacy-preserving server-less federated learning framework," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 7824–7838, 2025.
- [86] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "Pile: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
- [87] C. Fu et al., "Label inference attacks against federated learning," in *Proc. USENIX Secur. Symp.*, 2022, pp. 1397–1414.
- [88] H. Ding, J. Zhai, and Y. Nan, "Shiqing ma AIRTAG: Towards automated attack investigation by unsupervised learning with log texts," in *Proc. USENIX Secur. Symp.*, Anaheim, CA, USA, 2023, pp. 125–136.
- [89] I. Ghafir et al., "Detection of advanced persistent threat using machine-learning correlation analysis," *Future Gener. Comput. Syst.*, vol. 89, pp. 349–359, 2018.
- [90] A. Presekal, A. Stefanov, I. Semertzis, and P. Palensky, "Spatio-temporal advanced persistent threat detection and correlation for cyber-physical power systems using enhanced GC-LSTM," *IEEE Trans. Smart Grid*, vol. 16, no. 6, pp. 1654–1666, Mar. 2025.
- [91] L. Yang et al., "LLM-APTDs: A high-precision advanced persistent threat detection system for imbalanced data based on large language models with strong interpretability," *Future Gener. Comput. Syst.*, vol. 178, 2026, Art. no. 108315.



Kun Lan is currently working toward the Ph.D. degree with the School of Computer Science (School of Cyberspace Security and the School of Cryptography), Shanghai Jiao Tong University, Shanghai, China. Since 2005, he has been working with China Electronics Technology Cyber Security Company Ltd. He is also a Researcher-Level Senior Engineer conducting in-depth research in the field of cyberspace security. His research interests include cyber attack threat detection and deep learning.



Gaolei Li (Member, IEEE) is currently an Associate Professor with the School of Computer Science, Shanghai Jiao Tong University, Shanghai, China. He has authored or coauthored more than 100 papers, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY (IEEE TIFS), IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING (IEEE TDSC), NDSS, ACM CCS, ACM MM, and AAAI. His research interests include machine learning security and privacy-preserving. He was the recipient of the many awards, including the

Outstanding Paper Award of IEEE ISPA 2024, Third Prize of Science and Technology Progress Award of China Electric Power Development Acceleration Committee in 2024, Best Conference Paper Award of China Cryptology Society in 2020, and Best Conference Paper Award of IEEE CSIM Committee in 2018. He was also a TPC Member for CVPR 2024–2025, AAAI 2023&2024&2025, ACM MM 2023&2024&2025, and ICLR 2025. He is a Reviewer of IEEE TIFS, IEEE TDSC, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING.



Jianhua Li (Senior Member, IEEE) is currently a Professor/Ph.D. degree Supervisor and the Dean with the Institute of Cyber Science and Technology, Shanghai Jiao Tong University, Shanghai, China. He is also the Director with the National Engineering Laboratory for Information Content Analysis Technology, Director of Engineering Research Center for Network Information Security Management and Service of Chinese Ministry of Education. He has authored or coauthored more than 400 papers, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE

COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, ACM CCS, ACM MobiHoc, and IEEE Infocom. His research interests include information security, signal process, and computer network communication. He was the recipient of many awards, including Second Prize of National Technology Progress Award of China in 2005, First Prize of Science and Technology Progress Award of the Ministry of Education, First Prize of Science and Technology Progress Award of Shanghai, Best paper award of IEEE CSIM Committee in 2016, IEEE TETC in 2017, ESI Highly Cited Scientist from 2022 to 2024.



Wenkai Huang received the B.S. degree in information security (with IEEE Hons. Class) in 2023 from Shanghai Jiao Tong University, Shanghai, China, where he is currently working toward the Ph.D degree with the School of Computer Science. His research interests include backdoor learning, federated learning, and data security.



Yantao Yu (Member, IEEE) received the Ph.D. degree in information and communication engineering from Tongji University, Shanghai, China, in 2025. From 2023 to 2024, he was a Visiting Scholar at the Faculty of Computer Science, University of New Brunswick, Canada. He is currently a Postdoctoral Researcher with the School of Computer Science, Shanghai Jiao Tong University, Shanghai, China. His research interests mainly include security and privacy in Internet of Things, applied cryptography, and privacy computing.